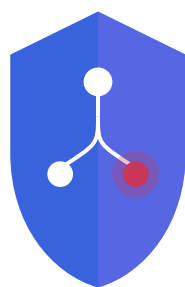


《系统安全综合设计与训练》

综合设计报告



AEGIS

基于抽象语法树结构指纹的 Web 攻击语义检测平台

*Semantic Web Attack Detection Platform
via SQL Abstract Syntax Tree Structural Fingerprinting*

课程名称 系统安全综合设计与训练

课程代码 60025127

选题类别 自拟题目（应用安全类）

适用专业 网络空间安全

完成形式 小组完成（4 人）

组 长 黄思凯

组 员 罗锦皓 曾铭 吴靖翔

指导教师 _____

计算机与信息学院 网络空间安全系

2026 年

摘要

以 SQL 注入为代表的注入类漏洞长期位居 OWASP Top 10 首位。当前主流的 Web 应用防火墙普遍采用基于正则表达式的特征匹配范式，其判定对象是请求载荷的文本形态。然而攻击者可通过 URL 多重编码、HTML 实体编码、注释拆分、数据库方言内联注释以及大小写混淆等手段，在不改变载荷语义的前提下任意改变其文本形态，致使特征匹配长期陷于“新绕过手法出现—补充规则—再被绕过”的被动循环。

针对上述问题，本文提出并实现了一种**基于抽象语法树结构指纹的语义检测方法**。该方法的核心观察是：注入攻击的本质目的是改变 SQL 的语义结构，而语义结构由语法树的形状唯一决定，与树中字面量的具体取值以及语句的文本书写形式无关。据此，本文设计了“词法分析—语法分析—字面量归一化—规范化序列化—哈希”的处理流水线，将任意 SQL 语句映射为一个仅反映其语义结构的定长指纹。理论分析与实验证明，该指纹同时具备参数无关性、形态无关性与结构敏感性三条性质，因而对编码混淆类绕过手法具有原理上的免疫能力。在指纹失配时，本文进一步提出基于路径签名多重集差分的结构定位算法，以 $O(n)$ 的复杂度精确定位注入发生的语法树节点，并将差分结果映射为具有安全语义的攻击手法分类，为安全取证与可视化提供了确定性依据。

在系统层面，本文设计了**安全网关与运行时应用自我保护（RASP）探针双层联动架构**。网关层工作于 HTTP 流量入口，其可见对象为请求参数文本，判定具有启发式性质；RASP 探针经字节码注入植入应用进程内部，钩取 JDBC 执行点，其可见对象为已完成拼接、即将送达数据库的真实语句，因而可执行确定性的结构比对，并在攻击生效前的最后时刻予以阻断。两层通过统一的追踪标识关联，可完整还原“HTTP 请求—污染参数—真实 SQL—结构变异—处置结果”的攻击证据链。此外，本文以同样的“语义优先于文本”的思路实现了基于文档对象模型解析与三级白名单的跨站脚本净化机制，并配套设计了令牌桶与滑动窗口双重限流、带时间衰减的来源信誉评估，以及基于哈希链的防篡改审计日志。

本文按照三轮测试法对系统进行了完整验证：第一轮在关闭防护并启用脆弱实现的条件 下确认漏洞真实存在，第二轮在开启防护的条件下验证检出与拦截能力，第三轮在启用安全实现 的条件下验证代码级修复的有效性。实验共设计 19 项测试用例，覆盖 OWASP Top 10（2021） 的七个类别。结果表明：网关可判定类攻击的实时拦截率为 12/12；全部 19 项漏洞的代码级 修复均验证有效；CVSS 评分不低于 7.0 的 18 项高危漏洞全部获得防护；审计日志哈希链完整 性校验通过；检测引擎单元测试通过率为 63/63，平均检测时延为 0.93 ms，显著优于 5 ms 的 设计指标。对比实验进一步表明，本方法对数据库方言内联注释等可绕过传统正则规则集的 载荷仍能稳定检出。

关键词： Web 应用防火墙；抽象语法树；结构指纹；语义检测；运行时应用自我保护；SQL 注入

Abstract

Injection flaws, exemplified by SQL injection, have persistently ranked at the top of the OWASP Top 10. Mainstream Web Application Firewalls predominantly adopt a regular-expression-based signature matching paradigm, whose object of adjudication is the *textual form* of a request payload. Adversaries, however, can arbitrarily alter this textual form—through multi-layer URL encoding, HTML entity encoding, comment splitting, dialect-specific inline comments, and case obfuscation—without altering the payload’s semantics. Signature matching is therefore trapped in a reactive cycle: a novel evasion emerges, a rule is patched, and the rule is bypassed again.

To address this problem, this work proposes and implements a **semantic detection method based on abstract syntax tree structural fingerprinting**. The central observation is that the essential objective of an injection attack is to *alter the semantic structure of the SQL statement*, and that this structure is uniquely determined by the shape of the syntax tree—independent of the concrete values of its literals and of the statement’s textual rendering. Accordingly, we design a pipeline of lexical analysis, syntactic analysis, literal normalization, canonical serialization, and hashing that maps an arbitrary SQL statement to a fixed-length fingerprint reflecting only its semantic structure. Both theoretical analysis and experiments establish that this fingerprint simultaneously satisfies *parameter invariance*, *form invariance*, and *structure sensitivity*, thereby conferring principled immunity to encoding-based evasion. Upon fingerprint mismatch, we further propose a structural localization algorithm based on *path-signature multiset differencing*, which identifies the precise syntax tree node at which injection occurred in $O(n)$ time and maps the resulting difference to security-semantic attack categories, providing a deterministic basis for forensics and visualization.

At the system level, we design a **dual-layer architecture coupling a security gateway with a Runtime Application Self-Protection (RASP) agent**. The gateway operates at the HTTP ingress, where only parameter text is observable and adjudication is necessarily heuristic. The RASP agent, injected into the application process via bytecode instrumentation, hooks the JDBC execution point and observes the *fully assembled statement about to reach the database*, thereby enabling deterministic structural comparison and interception at the last moment before the attack takes effect. The two layers are correlated by a unified trace identifier, reconstructing the complete evidence chain from HTTP request, through tainted parameter and actual SQL, to structural mutation and final verdict. Following the same principle of semantics over text, we implement a cross-site scripting sanitizer based on Document Object Model parsing with a three-tier allowlist, and complement the system with dual token-bucket and sliding-window rate limiting, time-decaying source reputation assessment, and a hash-chained tamper-evident audit log.

The system is validated through a three-round testing methodology: the first round confirms vulnerability reproducibility with protection disabled and vulnerable implementations active; the second verifies detection and interception with protection enabled; the third validates code-level remediation with secure implementations active. Nineteen test cases were designed, spanning seven categories of the OWASP Top 10 (2021). Results show a real-time interception rate of 12/12 for gateway-adjudicable attacks; code-level remediation verified effective for all 19 vulnerabilities; protection

achieved for all 18 high-severity vulnerabilities with $CVSS \geq 7.0$; successful integrity verification of the audit hash chain; a unit test pass rate of 63/63 for the detection engine; and a mean detection latency of 0.93 ms, substantially outperforming the 5 ms design target. Comparative experiments further demonstrate stable detection of payloads employing dialect-specific inline comments, which are capable of evading conventional regular-expression rule sets.

Keywords: Web Application Firewall; Abstract Syntax Tree; Structural Fingerprint; Semantic Detection; Runtime Application Self-Protection; SQL Injection

目录

摘 要	I
Abstract	I
目 录	IV
插图目录	IX
表格目录	X
1 绪论	1
1.1 研究背景与选题意义	1
1.2 国内外研究现状	1
1.2.1 基于规则匹配的检测方法	1
1.2.2 基于词法分析的检测方法	1
1.2.3 基于语法树验证的检测方法	2
1.2.4 基于机器学习的检测方法	2
1.2.5 运行时应用自我保护技术	2
1.2.6 现有方案的对比	2
1.3 本文主要工作与创新点	3
1.4 论文组织结构	4
2 需求分析	4
2.1 系统目标与设计约束	4
2.2 功能性需求	4
2.3 非功能性需求	4
2.4 威胁建模	4
2.4.1 数据流与信任边界	4
2.4.2 STRIDE 威胁识别	6
2.4.3 DREAD 风险定级	6
2.4.4 威胁分布分析	6
2.5 安全需求矩阵	6
2.6 可行性分析	7
3 系统设计	8
3.1 设计原则	8
3.2 总体架构	8
3.2.1 架构组成	8
3.2.2 双层联动架构的必要性论证	9
3.3 关键流程设计	10

3.3.1	请求检测主流程	10
3.3.2	基线学习与审核流程	10
3.4	数据模型设计	11
3.4.1	检测事件模型	11
3.4.2	基线模型	12
3.5	接口设计	12
3.6	部署架构	12
4	核心算法设计	13
4.1	算法总体流程	13
4.2	载荷多重归一化	13
4.2.1	设计动机	13
4.2.2	不动点解码	14
4.3	词法与语法分析	14
4.3.1	词法分析中的注释处理	14
4.3.2	容错语法分析	14
4.4	抽象语法树结构指纹	15
4.4.1	核心思想	15
4.4.2	形式化定义	15
4.4.3	指纹的关键性质	16
4.4.4	两处关键的工程决策	16
4.5	结构差分定位算法	17
4.5.1	问题与算法选型	17
4.5.2	路径签名多重集差分	17
4.5.3	变异的语义标注	17
4.5.4	算法描述	17
4.6	风险特征检测	19
4.6.1	设计动机	19
4.6.2	特征清单	19
4.7	综合风险评分模型	20
4.7.1	误报抑制机制	20
4.8	跨站脚本的语义净化	20
4.8.1	正则方案的失效	20
4.8.2	基于文档对象模型的三级白名单	20
4.8.3	协议判定的关键细节	20
5	安全设计与实现	21
5.1	纵深防御体系	21
5.2	认证与授权	22
5.2.1	令牌签名算法的强制约束	22

5.2.2	认证与授权的分离	22
5.3	敏感数据保护	22
5.3.1	口令存储	22
5.3.2	出站请求的地址约束	22
5.4	审计日志的不可否认性	22
5.4.1	哈希链设计	22
5.4.2	实现中的一处关键细节	23
5.5	可用性保障	23
5.5.1	失效安全策略	23
5.5.2	速率控制	23
5.5.3	来源信誉的时间衰减	24
5.6	安全编码实践	24
5.6.1	高危函数禁用	24
5.6.2	安全注释规范	24
5.6.3	受控漏洞的隔离	25
5.7	系统自身的攻击面分析	25
6	系统实现	25
6.1	开发环境与规模	25
6.2	检测引擎的实现	26
6.2.1	词法分析中的注释粘合	26
6.2.2	网关层参数判定的双探针法	27
6.3	RASP 探针的实现	27
6.3.1	钩取点选择	27
6.3.2	两处工程约束	27
6.4	可视化界面的实现	28
6.4.1	设计取向	28
6.4.2	语法树差分的渲染	28
6.5	受控靶场的实现	28
6.5.1	双实现对照结构	28
6.5.2	合规性措施	28
6.6	系统界面	28
7	实验与评估	29
7.1	实验设计	29
7.1.1	三轮测试方法	29
7.1.2	测试环境	30
7.1.3	测试用例设计	30
7.2	单元测试结果	31
7.3	三轮测试结果	31

7.3.1	总体结果	31
7.3.2	结果的可视化	31
7.3.3	高危漏洞的防护覆盖	32
7.4	绕过对抗实验	32
7.4.1	实验目的	32
7.4.2	实验设计	33
7.4.3	实验结果	33
7.4.4	与规则匹配方案的对比讨论	33
7.5	误报测试	33
7.5.1	实验设计	33
7.5.2	实验结果	33
7.5.3	开发过程中发现的两类误报及其修复	34
7.6	性能测试	34
7.6.1	检测时延	34
7.6.2	性能指标汇总	34
7.6.3	性能优化措施	35
7.7	结果分析与讨论	35
7.7.1	关于第一轮中的五项未复现用例	35
7.7.2	关于业务逻辑类漏洞的防线归属	36
8	项目管理	36
8.1	开发流程与进度安排	36
8.2	风险管理	37
8.3	工时统计	37
8.4	小组成员与分工	37
8.5	版本控制与过程留痕	38
8.6	质量保障措施	38
9	总结与展望	39
9.1	工作总结	39
9.2	创新点总结	39
9.3	局限性分析	39
9.4	未来改进方向	40
9.5	成员工作总结与心得	40
9.5.1	组长 黄思凯	40
9.5.2	组员 罗锦皓	41
9.5.3	组员 曾铭	41
9.5.4	组员 吴靖翔	41
	参考文献	43

附录	45
A 核心算法实现	45
B 测试用例载荷	46
C 项目成果清单	47
D 合规声明	47

插图

2.1	系统数据流图与信任边界划分	6
2.2	威胁分布特征分析	7
3.1	AEGIS 系统总体架构	9
3.2	注入攻击的检测与阻断时序	10
3.3	SQL 基线的学习与审核流程	11
4.1	结构指纹检测算法的处理流水线	13
4.2	合法查询与注入查询的语法树结构对比	15
5.1	哈希链式审计日志的防篡改机制	23
6.1	语法树分析页面的信息组织	29
7.1	三轮测试结果的可视化呈现	31
7.2	各检测路径的平均时延对比	35
8.1	项目开发进度甘特图	36
8.2	各阶段工时投入占比	37

表格

1.1	主流 Web 攻击检测方案的横向比较	3
2.1	功能性需求清单	5
2.2	非功能性需求指标	5
2.3	STRIDE 威胁清单、DREAD 定级与缓解措施	7
2.4	安全需求矩阵	8
3.1	系统模块职责与技术选型	9
3.2	网关层与探针层的可见性与判定性质对比	10
3.3	检测事件的关键字段设计	11
3.4	系统主要接口设计	12
3.5	服务部署与端口分配	12
4.1	常见编码混淆手法及其归一化结果	14
4.2	结构变异的语义分类规则	18
4.3	语法树层面的风险特征清单	19
4.4	跨站脚本净化的三级白名单模型	21
5.1	针对 SQL 注入的纵深防御层次	21
5.2	高危函数禁用清单与替代方案	24
6.1	开发环境与技术选型	26
6.2	各模块代码规模统计	26
6.3	RASP 探针的钩取点设计	27
6.4	脆弱实现与安全实现的代码对照示例	29
7.1	三轮测试法的实验设计	30
7.2	实验环境配置	30
7.3	测试用例的 OWASP 类别分布	30
7.4	检测引擎单元测试结果	31
7.5	三轮测试完整结果	32
7.6	绕过对抗实验结果	33
7.7	正常业务输入的误报测试	34
7.8	系统性能指标达成情况	35
7.9	性能优化措施及其效果	35
8.1	风险登记表与实际发生情况	37
8.2	小组成员分工与主要产出	38
B-1	主要测试用例的攻击载荷	46
C-1	项目交付成果	47

1 绪论

1.1 研究背景与选题意义

Web 应用已成为承载社会关键业务的基础设施，其安全性直接关系到数据资产与业务连续性。开放式全球应用安全项目（OWASP）在其发布的 2021 年度十大风险清单中，将注入（Injection）列为第三位风险，而在此前的 2013 与 2017 两个版本中，注入类漏洞长期位居首位 [1]。以 SQL 注入为代表的注入攻击之所以历经二十余年仍未消退，其根源在于一个结构性缺陷：应用程序将不可信的用户输入与可信的指令结构混合于同一文本通道中，致使解释器无法区分二者边界。

针对这一威胁，工业界形成了两条主要防线。其一是开发侧的根本性修复，即通过参数化查询将指令结构与数据彻底分离；其二是运行侧的旁路防护，即部署 Web 应用防火墙（Web Application Firewall, WAF）对请求流量实施检测与阻断。前者虽属治本之策，但在存量系统改造成本高昂、第三方组件不可控、开发规范执行不彻底等现实约束下，难以做到全覆盖；后者因而成为不可或缺的补充。

然而，当前主流 WAF 的技术路线存在一个值得深究的方法论问题。以 ModSecurity 核心规则集（Core Rule Set, CRS）为代表的产品，其判定依据是对请求载荷施加正则表达式匹配 [2]。该范式的判定对象是载荷的文本形态，而非其语义内涵。二者的错配构成了防护体系的根本性弱点：攻击者可通过 URL 多重编码、HTML 实体编码、注释拆分、数据库方言内联注释、大小写混淆等手段，在完全不改变载荷语义的前提下任意改变其文本形态。防护方因而被迫陷入“新绕过手法出现—补充规则—再被绕过”的被动循环，规则集规模持续膨胀而防护有效性并未同步提升，误报率反而随规则数量增长而恶化。

本课题正是针对上述方法论困境而设立。其核心追问是：能否找到一种检测依据，使其对攻击者可自由操纵的维度（文本形态）不敏感，而对攻击者必须改变的维度（语义结构）高度敏感？

1.2 国内外研究现状

1.2.1 基于规则匹配的检测方法

规则匹配是工业界应用最广的方案。ModSecurity CRS 通过数千条正则规则覆盖已知攻击特征，具备部署简便、可解释性强的优点 [2]。但如前所述，该方案的判定对象与攻击本质错配，且规则的通用性与精确性存在难以调和的矛盾：规则过宽则误报率高，过窄则易被变形绕过。文献 [3] 对该类方法的系统性评估表明，其对未知变形载荷的检出率显著低于对已知特征的检出率。

1.2.2 基于词法分析的检测方法

Galbreath 提出的 libinjection 是该方向的代表性工作 [4]。该方法将载荷切分为词法单元（token）序列，通过有限状态机识别注入特有的 token 组合模式。相较正则匹配，词法分析对大小写与空白等表层混淆具备天然免疫力，且检测速度极快。其局限在于仅停留于 token 序列

层面而未构建完整语法结构，因而难以表达“条件被剥离”“子查询嵌套”等需要层次信息才能刻画的语义变化，也无法定位攻击发生的具体结构位置。

1.2.3 基于语法树验证的检测方法

Buehrer 等人在 2005 年提出的 SQLGuard 是本课题最直接的理论渊源 [5]。该工作的核心洞察是：合法查询的语法树结构在运行时应保持稳定，注入则会引入结构变化；因此可在运行时比对“仅含开发者意图的查询”与“混入用户输入后的查询”两者的语法树，若不一致则判定为注入。Boyd 与 Keromytis 提出的 SQLrand 采用了另一种思路，通过对 SQL 关键字施加随机化编码，使攻击者注入的标准关键字无法被解析器识别 [6]。Halfond 与 Orso 提出的 AMNESIA 则结合静态分析与运行时监控，先由静态分析推导各查询点的合法 SQL 模型，再在运行时校验实际语句是否符合该模型 [7]。Su 与 Wassermann 从形式语言理论出发，将注入攻击形式化定义为“用户输入跨越了语法树的非终结符边界”，为该方法奠定了理论基础 [8]。

上述工作确立了“语义优于文本”的正确方向，但普遍存在三方面不足：其一，多数方案要求侵入式改造应用源码（如 SQLGuard 需开发者显式标注查询模板），部署代价高；其二，判定结果为二值的“匹配/不匹配”，无法定位注入发生的具体语法节点，对安全取证的支撑不足；其三，缺少面向工程落地的完整系统设计，如基线的自动学习与审核、检测结果的可视化呈现、以及与现有部署形态的兼容。

1.2.4 基于机器学习的检测方法

近年来，卷积神经网络 [9] 与人工神经网络 [10] 等方法被引入注入检测领域，在特定数据集上取得了较高的准确率。此类方法的优势在于可自动学习特征而无需人工构造规则，但也面临三重制约：模型判定过程缺乏可解释性，难以向安全运营人员说明“为何判定为攻击”；训练数据的分布偏差会直接转化为检测盲区；以及推理开销通常高于确定性算法，在高吞吐场景下形成瓶颈。文献 [11] 对该方向的综述亦指出，可解释性不足是其在生产环境落地的主要障碍。

1.2.5 运行时应用自我保护技术

运行时应用自我保护（Runtime Application Self-Protection, RASP）通过将检测逻辑植入应用进程内部，使防护能够观测到应用的真实运行时上下文 [12]。相较部署于流量入口的 WAF，RASP 的关键优势在于可见性：它能够观测到已完成拼接的真实 SQL，而非仅仅是 HTTP 参数。这一优势对判定的确定性具有决定性意义，构成了本文双层架构设计的直接依据。

1.2.6 现有方案的对比

表 1.1 从五个维度对上述方案进行了横向比较。可以看出，现有工作尚未形成一个同时具备“非侵入部署、确定性判定、结构级定位、可解释输出”四项特性的完整方案，这正是本课题的切入点。

表 1.1 主流 Web 攻击检测方案的横向比较

方案类别	判定对象	抗编码混淆	结构级定位	可解释性	部署侵入性
正则规则集 [2]	文本形态	弱	无	强	无
词法状态机 [4]	Token 序列	中	无	强	无
语法树验证 [5]	语法结构	强	无	强	高
关键字随机化 [6]	关键字编码	强	无	中	高
静态分析建模 [7]	语法模型	强	无	中	中
神经网络 [10]	统计特征	中	无	弱	无
本文方法	语法结构	强	有	强	无 ^a

^a 本文通过字节码注入实现探针植入，无需修改应用源码，故部署侵入性为“无”。

1.3 本文主要工作与创新点

本文围绕“以语义结构而非文本形态作为检测依据”这一核心思想，完成了从理论方法到工程系统的完整设计与实现。主要工作与创新点如下。

(1) 提出 SQL 抽象语法树结构指纹方法。 设计了“词法分析—语法分析—字面量归一化—规范化序列化—哈希”的处理流水线，将任意 SQL 语句映射为仅反映其语义结构的定长指纹。本文形式化证明了该指纹同时满足参数无关性、形态无关性与结构敏感性三条性质（详见第 4），因而对编码混淆类绕过手法具有原理上的免疫能力，而非依赖对已知手法的枚举。

(2) 提出基于路径签名多重集差分的结构定位算法。 针对现有语法树方案“只能判定是否变化、无法定位变化位置”的不足，本文摒弃复杂度过高的树编辑距离算法 [13]，转而采用路径签名多重集差分，以 $O(n)$ 的时间复杂度精确定位注入所在的语法树节点，并将差分结果映射为具有安全语义的攻击手法分类（恒真式绕过、联合查询注入、堆叠查询等），为取证与可视化提供确定性依据。

(3) 设计网关与 RASP 探针双层联动架构。 明确区分两层的可见性差异：网关层仅能观测 HTTP 参数文本，判定必然是启发式的；探针层可观测已完成拼接的真实 SQL，判定是确定性的。两层通过统一追踪标识关联，可完整还原攻击证据链。该架构在保持非侵入部署的同时，兼顾了拦截时效性与判定准确性。

(4) 实现完整的工程系统并进行严格验证。 系统包含检测引擎、RASP 探针、安全网关、控制台服务、受控靶场与可视化前端六个模块，代码规模约 1.7 万行。采用三轮测试法进行验证：分别确认漏洞真实存在、防护检出有效、代码修复有效。设计 19 项测试用例覆盖 OWASP Top 10 的七个类别，全部高危漏洞获得防护，平均检测时延 0.93 ms。

1.4 论文组织结构

本文余下部分组织如下。第 2 进行需求分析，采用 STRIDE 方法建立威胁模型并以 DREAD 模型完成风险定级。第 3 阐述系统总体设计，重点论证双层架构的必要性。第 4 是本文的技术核心，给出结构指纹与结构差分算法的形式化定义、性质证明与复杂度分析。第 5 论述系统自身的安全设计，包括纵深防御体系、防篡改审计与安全编码实践。第 6 介绍关键模块的实现细节。第 7 报告实验设计与评估结果，包含功能验证、绕过对抗实验、误报测试与性能测试。第 8 记录项目管理过程。第 9 总结全文，并诚实讨论方法的局限性与后续改进方向。

2 需求分析

2.1 系统目标与设计约束

本系统的总体目标是：在不修改被保护应用源代码的前提下，对 Web 应用面临的注入类攻击实施高准确率的实时检测与阻断，并为安全运营提供具备结构级定位能力的取证依据。

围绕这一目标，本文明确以下四项设计约束，它们贯穿后续的架构与算法设计。

约束 C1（判定依据的正交性） 检测依据须对攻击者可自由操纵的维度（文本形态）不敏感，而对攻击者必须改变的维度（语义结构）高度敏感。

约束 C2（部署的非侵入性） 防护能力的植入不得要求修改应用源码或业务逻辑，以适应存量系统与第三方组件的改造限制。

约束 C3（失效安全） 检测组件自身的任何异常均不得阻断正常业务流量，即在可用性与检测完整性冲突时优先保障可用性。

约束 C4（判定的可解释性） 每一次拦截决策须能向安全运营人员说明“依据何种结构证据判定为攻击”，而非仅输出一个置信度分值。

2.2 功能性需求

依据上述目标，本文梳理出 20 项功能性需求，按所属子系统归类列于表 2.1。优先级中 P0 表示核心功能，P1 表示重要功能，P2 表示增强功能。

2.3 非功能性需求

表 2.2 给出量化的非功能性指标。其中检测时延指标依据如下考量确定：Web 请求的端到端时延通常在数十毫秒量级，安全检测作为旁路环节，其开销应控制在总时延的 10% 以内方可认为对用户体验无感知，故设定 5 ms 的上限。

2.4 威胁建模

2.4.1 数据流与信任边界

威胁建模的第一步是识别系统的数据流与信任边界。图 2.1 给出了本系统的数据流图。图中虚线表示信任边界，跨越边界的数据流是威胁分析的重点对象。

表 2.1 功能性需求清单

编号	子系统	需求描述	优先级
FR-01	网关	反向代理转发，支持常见 HTTP 方法与表单、JSON 载荷	P0
FR-02	网关	载荷多重归一化：URL、HTML 实体、Unicode 解码至不动点	P0
FR-03	网关	请求参数的攻击意图启发式判定	P0
FR-04	网关	令牌桶与滑动窗口双重速率限制	P0
FR-05	网关	来源信誉评估与动态封禁，评分随时间衰减	P1
FR-06	网关	注入追踪标识，实现与探针层的事件关联	P0
FR-07	引擎	SQL 词法分析，消解注释拆分与大小写混淆	P0
FR-08	引擎	递归下降语法分析，构建抽象语法树	P0
FR-09	引擎	字面量归一化与规范化序列化	P0
FR-10	引擎	结构指纹计算，指纹与接口标识绑定	P0
FR-11	引擎	路径签名多重集差分与变异节点语义标注	P0
FR-12	引擎	风险特征检测，支撑无基线场景的冷启动判定	P0
FR-13	引擎	基于文档对象模型解析的跨站脚本净化	P0
FR-14	引擎	路径穿越与命令注入检测	P1
FR-15	探针	字节码注入，钩取数据库执行与命令执行入口	P0
FR-16	探针	在语句送达数据库前完成结构判定与阻断	P0
FR-17	探针	基线自动学习，按结构指纹去重	P1
FR-18	控制台	事件汇聚、哈希链审计与完整性校验	P0
FR-19	控制台	统计聚合与实时推送	P0
FR-20	前端	语法树差分可视化与攻击证据链呈现	P0

表 2.2 非功能性需求指标

编号	属性	指标
NFR-01	检测时延	单次结构检测的平均时延不高于 5 ms
NFR-02	吞吐能力	单实例不低于 500 QPS（本地验证环境）
NFR-03	误报率	正常业务流量的误报率低于 1%
NFR-04	漏报率	对测试载荷集的漏报率为 0
NFR-05	呈现时延	攻击发生至可视化界面呈现不超过 500 ms
NFR-06	可用性	检测组件异常时降级放行，不阻断业务
NFR-07	可维护性	检测引擎单元测试行覆盖率不低于 70%

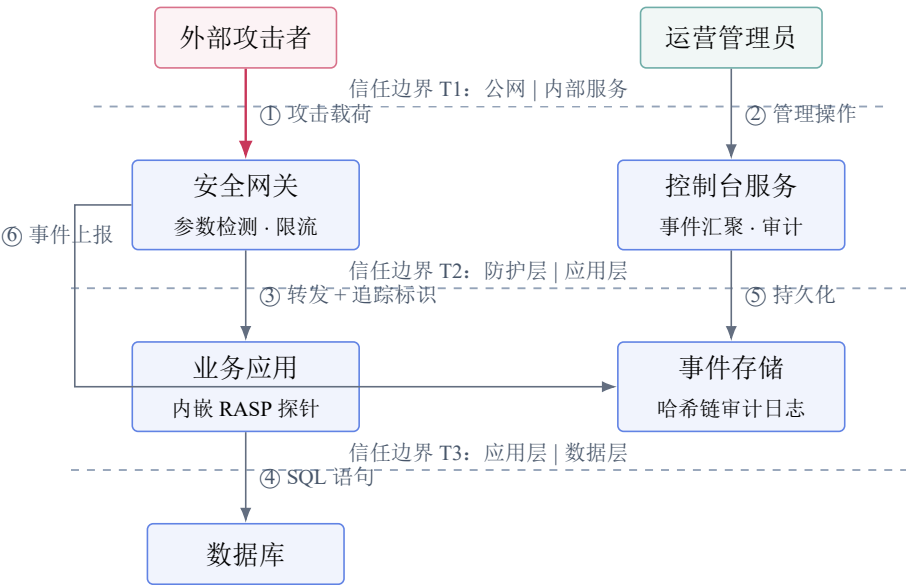


图 2.1 系统数据流图与信任边界划分

2.4.2 STRIDE 威胁识别

本文采用 STRIDE 方法 [14] 对跨越信任边界的数据流进行系统性威胁识别。STRIDE 是六类威胁英文首字母的缩写：假冒（Spoofing）、篡改（Tampering）、抵赖（Repudiation）、信息泄露（Information Disclosure）、拒绝服务（Denial of Service）与权限提升（Elevation of Privilege）。

2.4.3 DREAD 风险定级

对识别出的威胁，本文采用 DREAD 模型进行量化定级。该模型从潜在危害（Damage）、可复现性（Reproducibility）、可利用性（Exploitability）、受影响用户范围（Affected Users）与可发现性（Discoverability）五个维度各赋 0 ~ 10 分，取算术平均作为综合风险分。参照 CVSS 3.1 的分级惯例 [15]，综合分不低于 7.0 者判定为高危，须在设计阶段给出明确的缓解措施并在测试阶段提供修复证明。

表 2.3 给出完整的威胁清单与缓解措施映射。

2.4.4 威胁分布分析

图 2.2 从两个维度呈现威胁的分布特征。左图显示，篡改类威胁（T-01、T-02、T-07、T-11）数量最多，共 4 项，占威胁总数的三分之一，这与注入类攻击是 Web 应用主要威胁的行业观察一致 [1]；右图显示，风险分不低于 7.0 的高危及以上威胁共 9 项，占比 75%，表明本系统面对的威胁整体处于高风险水平，防护设计须以此为基准。

2.5 安全需求矩阵

将上述威胁分析结果映射为具体的安全属性需求，得到表 2.4。该矩阵建立了“安全属性—威胁编号—实现机制”的三元对应关系，为后续设计提供了可追溯的依据。

表 2.3 STRIDE 威胁清单、DREAD 定级与缓解措施

编号	类型	威胁描述	DREAD	等级	缓解措施
T-01	T	SQL 注入篡改查询逻辑	9.0	严重	语法树结构指纹+探针确定性判定+参数化查询
T-02	T	跨站脚本注入恶意脚本	8.0	高	文档对象模型解析+三级白名单净化
T-03	S	令牌伪造与算法混淆	8.0	高	强制签名算法白名单, 拒绝无签名令牌
T-04	E	水平与垂直越权访问	8.0	高	资源属主强校验+基于角色的访问控制
T-05	I	路径穿越读取任意文件	7.5	高	路径规范化后的根目录前缀判定
T-06	D	高频请求耗尽服务能力	7.0	高	令牌桶+滑动窗口+来源信誉封禁
T-07	T	命令注入执行系统指令	9.0	严重	探针钩取执行点+参数数组化调用
T-08	I	异常信息泄露内部结构	6.0	中	全局异常处理, 对外返回通用提示
T-09	R	攻击者否认其攻击行为	5.0	中	哈希链审计日志+全链路追踪标识
T-10	I	服务端请求伪造探测内网	7.0	高	域名解析后的地址判定+内网网段拒绝
T-11	T	不安全反序列化	8.0	高	禁用原生反序列化, 改用类型白名单
T-12	S	编码混淆绕过检测	8.0	高	多重归一化+语法结构判定(核心)

类型列缩写: S 假冒, T 篡改, R 抵赖, I 信息泄露, D 拒绝服务, E 权限提升。

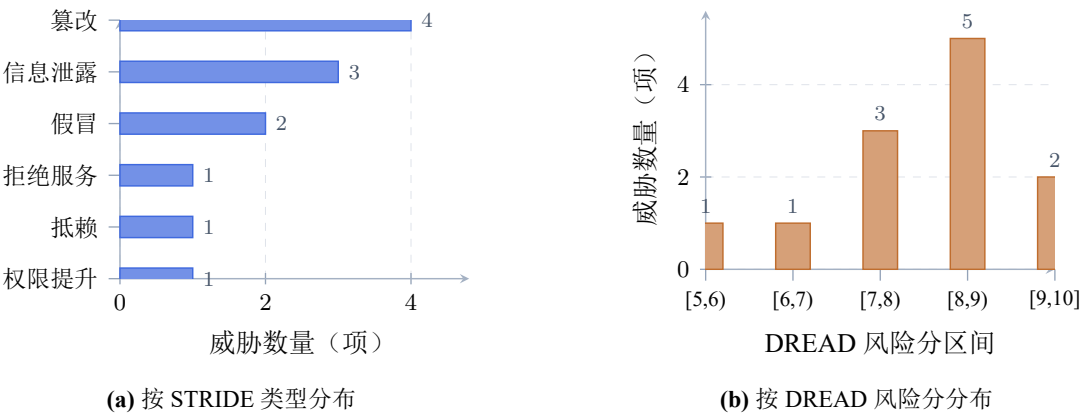


图 2.2 威胁分布特征分析

2.6 可行性分析

技术可行性。本文方法的三项关键技术均有成熟基础：SQL 的语法分析属编译原理的经典问题 [16]，递归下降法实现直观且便于施加容错策略；Java 平台的字节码注入有 Byte Buddy 等成熟工具支撑 [17]；语法树差分虽有树编辑距离等复杂算法 [13, 18]，但本文所需的“定位新增与缺失结构”可由多重集差分以线性复杂度完成。

进度可行性。课程设计周期为两周。本文将风险最高的字节码注入验证前置到第三日完成，并预设了“退化为数据访问层显式埋点”的备选方案，以控制技术不确定性对进度的影响。

表 2.4 安全需求矩阵

安全属性	对应威胁	实现机制
机密性	T-05, T-08, T-10	凭据加盐哈希存储；日志敏感字段脱敏；出站地址白名单
完整性	T-01, T-02, T-07, T-11	结构指纹比对；语义净化；参数化查询与参数数组化调用
可用性	T-06	双重速率限制；检测异常降级放行
可认证性	T-03	签名算法白名单；令牌短时效与唯一标识
可授权性	T-04	资源属主校验；基于角色的访问控制
可审计性	T-09	全链路追踪标识；结构化事件记录
不可否认性	T-09	哈希链式审计日志与完整性校验接口

法律合规性。 本课题涉及漏洞构造与攻击测试，须严格遵守法律边界。所有测试均在本地隔离环境中针对自建靶场执行，不涉及任何第三方系统；靶场中的漏洞代码集中隔离并显式标注，不随生产构建发布；反序列化类测试采用无害化载荷。相关实践符合《中华人民共和国网络安全法》《数据安全法》以及国家标准 GB/T 20984 关于风险评估活动的规范要求 [19]。

3 系统设计

3.1 设计原则

系统设计遵循以下四项安全工程原则，它们与 GB/T 25070 关于安全设计技术要求的规定保持一致 [20]。

- **纵深防御：**不依赖任何单点防护。同一威胁由网关层、探针层与应用代码层分别施加约束，任一层的失效不导致整体防护失守。
- **最小权限：**探针仅钩取必要的执行点；控制台按角色划分操作权限；靶场的文件访问严格限定于指定根目录之下。
- **失效安全：**检测组件的异常一律降级为放行并记录，而非阻断业务。这一取舍的依据是：防护组件的可用性故障若转化为业务中断，其危害往往超过被防护的威胁本身。
- **默认拒绝：**跨站脚本净化与出站地址判定均采用白名单机制，未显式允许者一律拒绝。相较黑名单，白名单对未知攻击手 □□ 具有天然的覆盖能力。

3.2 总体架构

3.2.1 架构组成

系统采用分层架构，由六个模块构成，如图 3.1 所示。其中检测引擎（aegis-core）作为共享库同时被网关与探针依赖，保证两层的判定逻辑严格一致，避免因实现差异导致的防护盲区。

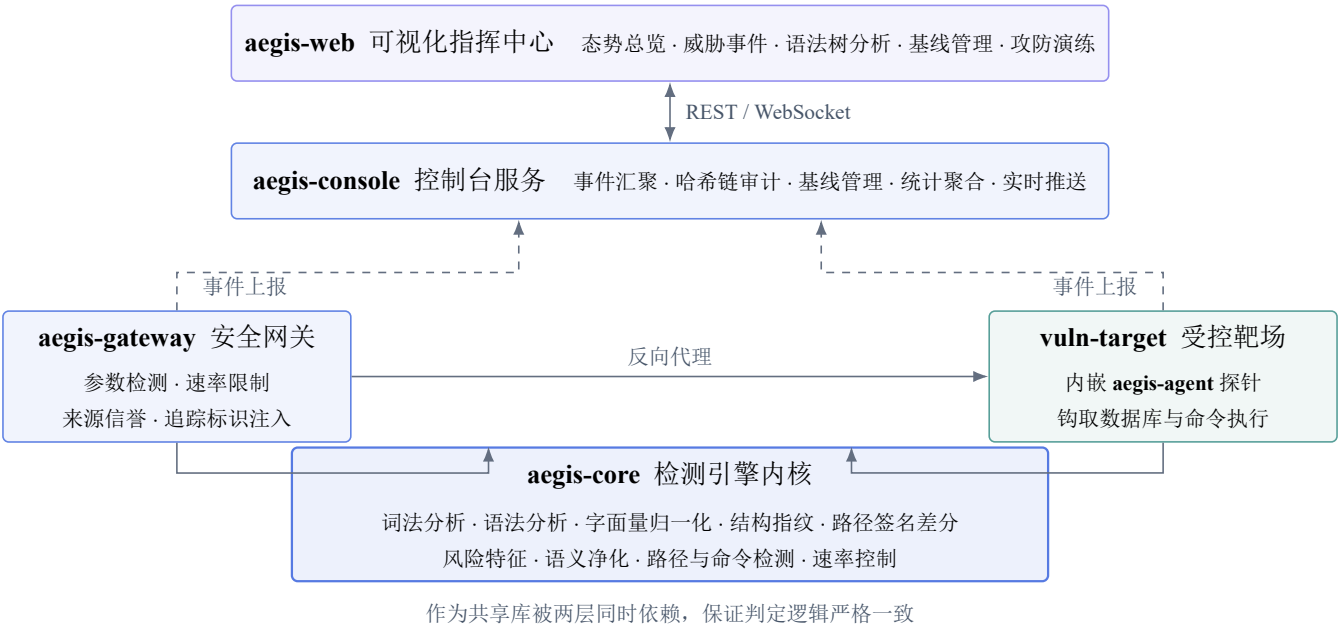


图 3.1 AEGIS 系统总体架构

各模块的职责与技术选型汇总于表 3.1。

表 3.1 系统模块职责与技术选型

模块	形态	核心职责	关键技术
aegis-core	共享库	检测算法内核	自研词法与语法分析器
aegis-agent	Java Agent	运行时钩取与确定性判定	Byte Buddy 字节码注入
aegis-gateway	服务	反向代理与启发式预判	Servlet 过滤器链
aegis-console	服务	事件汇聚与审计存证	Spring Boot, JPA
vuln-target	服务	受控靶场	双实现对照结构
aegis-web	单页应用	可视化与交互	Vue 3, AntV G6

3.2.2 双层联动架构的必要性论证

本文的架构核心在于网关层与探针层的分工。这一设计并非出于工程便利，而是源于二者在可见性上的本质差异。以下通过一个具体场景予以说明。

考虑请求参数 keyword = admin。该值是否构成攻击？

网关层的困境在于：它只能观测到这个字符串本身，无法确知该参数将被拼接进何种上下文。若后端使用参数化查询，则该值绝无危害；若后端拼接进 LIKE 子句，它仍是正常查询；唯有当它被拼接进特定位置并配合闭合符号时才可能构成注入。网关缺少判定所需的下游上下文，因此其判定必然是启发式的，在误报与漏报之间只能取折中。

探针层则处于完全不同的位置。它钩取的是数据库驱动的执行入口，此刻 SQL 已完成全部拼接。对这样一条已成形的语句，判定不再是“像不像攻击”的概率问题，而是“语法结构是否发生变化”的确定性问题。这一差异可归纳为表 3.2。

表 3.2 网关层与探针层的可见性与判定性质对比

维度	网关层	探针层
部署位置	HTTP 流量入口	应用进程内部
可见对象	请求参数文本	已拼接的真实语句
上下文信息	缺失下游上下文	具备完整执行上下文
判定性质	启发式预判	确定性结构比对
误报可能	存在	极低
拦截时机	请求进入前	语句执行前
主要价值	快速阻断、速率控制	精确取证、结构定位

两层的价值互补：网关层以低成本快速滤除明显攻击并承担速率控制职责，探针层对通过网关的流量施加确定性校验。二者通过统一的追踪标识关联，共同构成完整的证据链。

3.3 关键流程设计

3.3.1 请求检测主流程

图 3.2 以时序图形式给出一次攻击请求的完整处理流程。需特别关注第 □ 步：探针在语句送达数据库之前抛出安全异常，使攻击在产生任何实际效果前即被阻断。

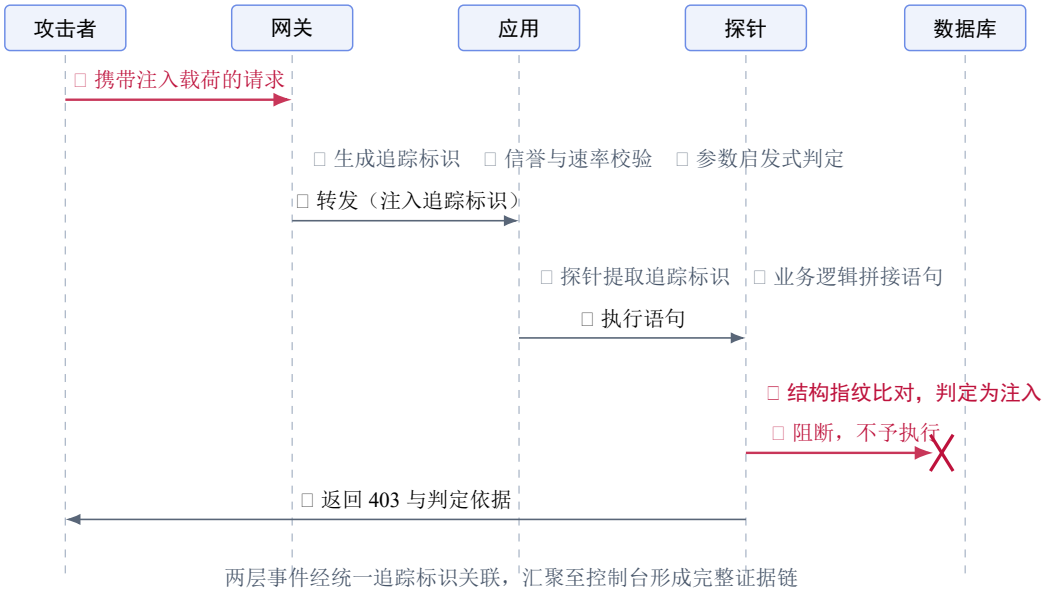


图 3.2 注入攻击的检测与阻断时序

3.3.2 基线学习与审核流程

结构指纹的比对需要基线作为参照。基线的建立采用“自动学习、人工确认”的两阶段流程，如 图 3.3 所示。

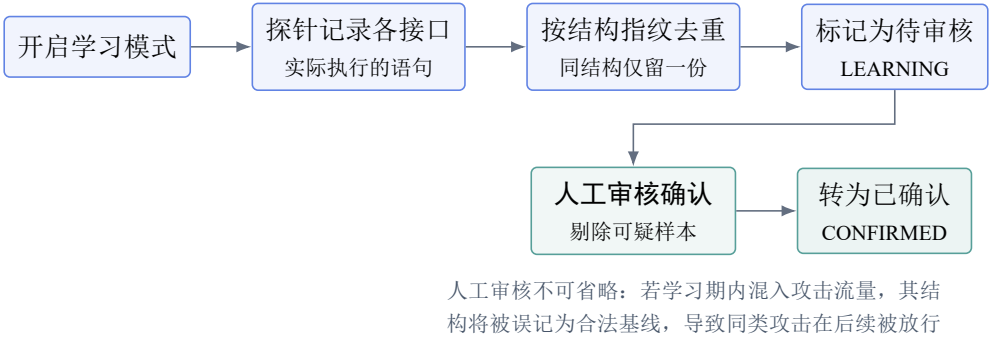


图 3.3 SQL 基线的学习与审核流程

注 3.1. 基线为空时系统并不失效。此时检测退化为风险特征判定模式，依据恒真式、集合运算、危险函数调用等“无论何种业务场景都不应出现”的绝对特征进行判断，保证系统开箱即用。该机制的细节见 第 4.6。

3.4 数据模型设计

3.4.1 检测事件模型

检测事件是贯穿系统的核心数据实体。其字段设计需同时满足三方面要求：支撑实时展示、支撑事后取证、支撑完整性校验。表 3.3 给出关键字段及其用途。

表 3.3 检测事件的关键字段设计

字段	类型	用途说明
event_id	UUID	事件唯一标识
trace_id	UUID	关联同一请求在两层产生的事件
layer	枚举	事件产生层次：网关 / 探针
threat_type	枚举	威胁类型，对应 OWASP 分类
severity	枚举	严重级别，与 CVSS 区间对齐
verdict	枚举	实际处置：放行 / 记录 / 拦截
raw_sql	文本	探针捕获的真实语句
baseline_fp	字符串	基线结构指纹
actual_fp	字符串	实际结构指纹
similarity	浮点	结构相似度（Jaccard 系数）
diff_json	文本	结构差分结果与可视化数据
risk_score	整型	综合风险评分
prev_hash	字符串	前一条记录的哈希，构成链式依赖
hash	字符串	本条记录的哈希

其中 trace_id 与哈希链字段是两处关键设计。前者是双层架构得以形成完整证据链的技术基础；后者为审计日志提供不可否认性，其原理详见 第 5.4。

3.4.2 基线模型

基线实体记录各接口的合法结构指纹。除指纹本身外，还保存归一化后的语句模板与语法树的可视化数据：前者供人工审核时判读，后者供前端渲染结构预览。状态字段区分学习中、已确认与已禁用三种状态，构成基线的生命周期管理。

3.5 接口设计

系统对外接口按用途分为三类，如表 3.4 所示。其中内部上报接口采用共享密钥认证，防止外部伪造安全事件以掩盖真实攻击或制造告警噪声。

表 3.4 系统主要接口设计

类别	接口	说明
查询类	GET /api/events	事件分页查询，支持多维过滤
	GET /api/events/trace/{id}	按追踪标识还原完整证据链
	GET /api/events/verify-chain	审计日志完整性校验
	GET /api/stats/*	统计聚合，供大屏展示
管理类	PUT /api/policy	切换防护模式与检测参数
	GET /api/baselines	基线列表与结构预览
	POST /api/baselines/{id}/confirm	基线人工确认
内部类	POST /internal/agent/report	探针与网关事件上报
	WS /ws/aegis	实时事件推送通道

3.6 部署架构

系统采用分端口部署，各服务职责边界清晰。演示与测试时可通过访问不同端口对比“有防护”与“无防护”两种状态下的系统行为，这一设计直接支撑了第 7 中的三轮测试方法。端口分配见表 3.5。

表 3.5 服务部署与端口分配

端口	服务	用途
5173	可视化前端	指挥中心界面
8080	控制台服务	事件汇聚与管理接口
8000	安全网关	受防护入口，演示时访问此端口
8090	受控靶场	直连入口，用于无防护状态对比

4 核心算法设计

本章是全文的技术核心。第 4.1 给出算法总体流程；第 4.2 至第 4.4 依次阐述归一化、语法分析与结构指纹的构造，并形式化证明指纹的三条关键性□□；第 4.5 提出结构差分定位算法；第 4.6 与第 4.7 分别给出风险特征检测与综合评分模型；第 4.8 阐述跨站脚本的语义净化方法。

4.1 算法总体流程

检测流程如图 4.1 所示。载荷首先经过多重归一化以消解编码混淆，随后由词法与语法分析构建抽象语法树；语法树经字面量归一化与规范化序列化后计算结构指纹；指纹与基线比对，命中则放行，失配则进入结构差分定位与风险评分。

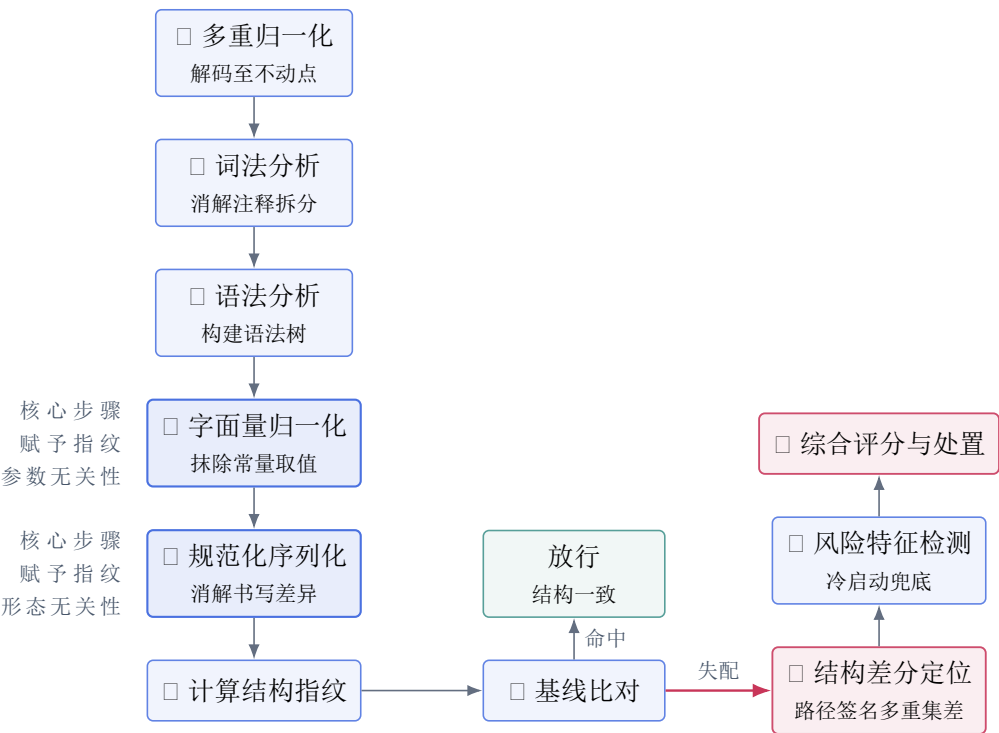


图 4.1 结构指纹检测算法的处理流水线

4.2 载荷多重归一化

4.2.1 设计动机

编码是攻击者规避文本匹配最直接的手段。表 4.1 列举了常见的混淆形式。若不做归一化，任何后续检测均可被单次 URL 编码轻易绕过。

表 4.1 常见编码混淆手法及其归一化结果

混淆手法	示例	归一化后
URL 单重编码	%27%20OR%201%3D1	' OR 1=1
URL 双重编码	%2527%2520OR	' OR
HTML 实体编码	' OR 1=1	' OR 1=1
Unicode 转义	\u0027 OR	' OR
全角字符	' O R 1 = 1	' OR 1=1
方言内联注释	/*!50000UNION*/	UNION
空白与控制字符	OR\t\n 1=1	OR 1=1
大小写混淆	oR uNiOn	or union

4.2.2 不动点解码

单次解码不足以应对多重编码。本文采用循环解码至不动点的策略：反复施加解码算子直至内容不再变化。设 D 为复合解码算子， s_0 为原始载荷，则解码序列为

$$s_{k+1} = D(s_k), \quad k = 0, 1, 2, \dots$$

(4.1)

终止条件为 $s_{k+1} = s_k$ （达到不动点）或 $k \geq K_{\max}$ 。

注 4.1 (解码深度上限的必要性). $K_{\max}$ 的设置并非工程妥协，而是必要的安全措施。攻击者可构造极深的嵌套编码，使解码过程消耗大量计算资源，形成拒绝服务攻击（对应威胁 T-06）。本文取 $K_{\max} = 5$ ，同时将解码深度本身作为风险信号：正常业务请求几乎不会出现三重以上编码，故 $k \geq 3$ 时额外累加风险分。

4.3 词法与语法分析

4.3.1 词法分析中的注释处理

词法分析阶段需处理一个易被忽视的细节：数据库方言的可执行注释。MySQL 支持形如 `/*!50000UNION*/` 的语法，其内容会被真实执行。若词法器将其作为普通注释整体丢弃，攻击载荷将退化为无害文本，造成严重漏报。

因此本文对两类注释采取不同策略：普通注释 `/*...*/` 识别为注释单元并在后续过滤中丢弃，使被拆分的关键字重新相邻；可执行注释 `/*!...*/` 则递归切分其内容并作为正常词法单元加入序列，同时标记来源以供风险评估。

4.3.2 容错语法分析

攻击载荷常导致 SQL 语法非法。若解析器遇错即抛出异常，检测将随之失效——解析失败本身不能成为绕过手段。为此，本文的递归下降解析器采用容错策略：无法识别的片段封装为未知节点并继续解析，保证任何输入都能得到一棵可分析的树。同时设置递归深度上限，防御深层嵌套构造引发的栈溢出。

4.4 抽象语法树结构指纹

4.4.1 核心思想

结构指纹方法建立于如下观察之上：

一条 SQL 语句的**语义结构**由其语法树的形状唯一决定，与树中字面量的具体取值无关，也与语句的文本书写形式无关。而注入攻击的本质目的正是**改变语义结构**。

图 4.2 直观展示了这一观察。左侧两条语句仅参数取值不同，其语法树完全同构；右侧语句因注入而在 WHERE 子树中新增了一个 OR 节点，结构发生实质变化。

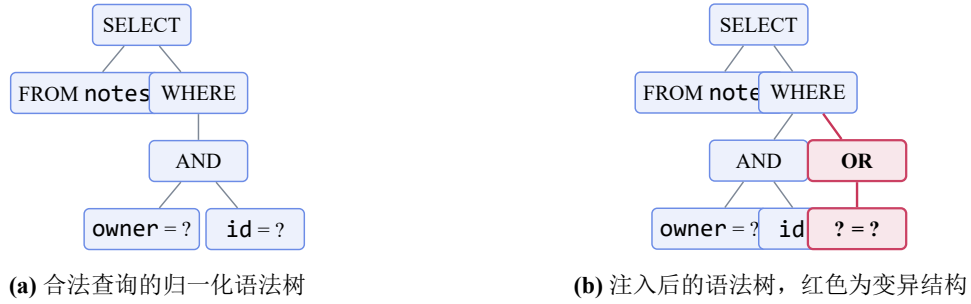


图 4.2 合法查询与注入查询的语法树结构对比

4.4.2 形式化定义

定义 4.1 (抽象语法树). 设 Σ_N 为节点类型集合, Σ_V 为节点值域。抽象语法树 $T = (V, E, \tau, \nu)$ 是一棵有根有序树, 其中 V 为节点集, E 为边集, $\tau: V \rightarrow \Sigma_N$ 为类型映射, $\nu: V \rightarrow \Sigma_V \cup \{\perp\}$ 为值映射。

定义 4.2 (字面量归一化算子). 归一化算子 $\mathcal{N}: T \rightarrow T'$ 将语法树中所有字面量节点的值替换为统一占位符 $\square$, 同时保留表名、列名、函数名与运算符:

$$\nu'(v) = \begin{cases} \square, & \tau(v) = \text{LITERAL} \wedge \nu(v) \notin \mathcal{L}_{\text{sem}} \\ \nu(v), & \text{其他} \end{cases} \quad (4.2)$$

其中 $\mathcal{L}_{\text{sem}} = \{\text{NULL}, \text{TRUE}, \text{FALSE}\}$ 为参与逻辑判断、不可归一化的语义字面量。

定义 4.3 (结构指纹). 设 $\mathcal{S}$ 为规范化序列化算子, H 为密码学哈希函数, e 为接口标识。语句 q 在接口 e 上的结构指纹定义为

$$\mathcal{F}(q, e) = H(e \parallel \mathcal{S}(\mathcal{N}(\text{parse}(q)))) \quad (4.3)$$

其中 $\parallel$ 表示字符串连接。

注 4.2 (指纹绑定接口标识的理由). 式 (4.3) 中将接口标识 e 纳入哈希输入并非冗余。若指纹全局共享, 攻击者可将接口 A 的合法结构用于接口 B 以绕过检测。绑定 e 后基线成为接口级隔离, 此类跨接口复用攻击自然失效。

4.4.3 指纹的关键性质

性质 4.1 (参数无关性). 设 q_1, q_2 为两条语句, 若二者的语法树在字面量归一化后同构, 则 $\mathcal{F}(q_1, e) = \mathcal{F}(q_2, e)$ 。

证明. 由归一化算子的定义, 所有非语义字面量被替换为同一占位符 $\square$ 。若 $\mathcal{N}(T_1) \cong \mathcal{N}(T_2)$, 则二者的规范化序列化结果相同, 即 $\mathcal{S}(\mathcal{N}(T_1)) = \mathcal{S}(\mathcal{N}(T_2))$ 。代入式 (4.3) 即得 $\mathcal{F}(q_1, e) = \mathcal{F}(q_2, e)$ 。□

性质 4.2 (形态无关性). 设 q 为一条语句, ϕ 为任意不改变语义的文本变换 (含大小写变换、空白调整、注释插入、编码变换), 则 $\mathcal{F}(\phi(q), e) = \mathcal{F}(q, e)$ 。

证明. 分两步说明。其一, 编码类变换由多重归一化在解析前消解, 故解析器接收到的输入相同。其二, 大小写、空白与注释类变换由词法分析消解: 关键字统一大写、标识符统一小写、空白折叠、普通注释丢弃。因此二者产生相同的词法单元序列, 进而构建同构的语法树, 由性质 4.1 即得结论。□

性质 4.3 (结构敏感性). 若语句 q' 相对 q 引入了新的语法节点或删除了原有节点 (即 $\mathcal{N}(T') \not\cong \mathcal{N}(T)$), 则在哈希函数抗碰撞的前提下, $\mathcal{F}(q', e) \neq \mathcal{F}(q, e)$ 以压倒性概率成立。

证明. 规范化序列化算子 $\mathcal{S}$ 是单射的: 不同构的归一化语法树产生不同的序列化字符串 (因序列化保留了完整的节点类型与层次信息)。故 $\mathcal{S}(\mathcal{N}(T')) \neq \mathcal{S}(\mathcal{N}(T))$ 。由哈希函数的抗碰撞性, 二者哈希值相等的概率可忽略。□

性质 4.2 与性质 4.3 共同构成了本方法的理论基础: 攻击者可自由操纵文本形态 (指纹不变), 但一旦改变语义结构 (这正是注入的目的), 指纹必然改变。二者的正交性使得编码混淆类绕过在原理上失效, 而非依赖对已知手法的枚举。

4.4.4 两处关键的工程决策

形式化定义之外, 实际实现中有两处细节直接影响方法的可用性, 值得专门说明。

IN 列表的长度无关化。 考虑 `col IN (1,2)` 与 `col IN (1,2,3)`。二者业务语义相同, 仅参数个数不同。若不做处理, 每种列表长度都会产生独立指纹, 导致基线数量随参数个数线性膨胀 (即基线爆炸)。本文在归一化阶段将连续的占位符折叠为单个, 使 `IN (?, ?, ?)` 收敛为 `IN (?)`。

函数名的保留。 一个易犯的错误是将函数名也纳入归一化。但 `SLEEP`、`BENCHMARK`、`LOAD_FILE`、`EXTRACTVALUE` 等函数是时间盲注与带外注入的核心标志。若将其归一化, 最危险的一类攻击将完全漏报。因此本文明确保留函数名, 仅归一化其参数。

4.5 结构差分定位算法

4.5.1 问题与算法选型

指纹比对只能回答“结构是否变化”，无法回答“变在何处”。后者对安全取证至关重要——运营人员需要知道攻击注入在语法树的哪个位置，方能理解攻击意图并评估影响范围。

树结构的差异度量有成熟的理论工具，即树编辑距离 [13]。但经典的 Zhang-Shasha 算法复杂度为 $O(n^2 \cdot d^2)$ (d 为树深)，在网关热路径上不可接受 [18]。

本文注意到：安全检测并不需要最小编辑脚本，只需知道哪些结构是新增的、哪些是缺失的。这一弱化的需求允许采用复杂度低得多的方案。

4.5.2 路径签名多重集差分

定义 4.4 (节点路径签名). 节点 v 的路径签名定义为从根到 v 的类型路径与节点自身特征的拼接：

$$\sigma(v) = \sigma(\text{parent}(v)) \parallel \tau(v) \parallel \kappa(v) \quad (4.4)$$

其中 $\kappa(v)$ 为节点的判别特征。对函数调用、表引用、列引用与运算符节点， $\kappa(v) = \nu(v)$ ；对已归一化的字面量节点， $\kappa(v) = \varepsilon$ 。

定义 4.5 (签名多重集). 语法树 T 的签名多重集为 $\Sigma(T) = \{\{\sigma(v) : v \in V(T)\}\}$ 。

设基线树为 T_b 、实际树为 T_a ，则新增结构与缺失结构分别为多重集差：

$$\Delta^+ = \Sigma(T_a) \setminus \Sigma(T_b), \quad \Delta^- = \Sigma(T_b) \setminus \Sigma(T_a) \quad (4.5)$$

结构相似度采用 Jaccard 系数度量：

$$J(T_b, T_a) = \frac{|\Sigma(T_b) \cap \Sigma(T_a)|}{|\Sigma(T_b) \cup \Sigma(T_a)|} \quad (4.6)$$

命题 4.1 (复杂度). 签名多重集的构建与差分运算的时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ ，其中 $n = |V(T_a)| + |V(T_b)|$ 。

证明. 签名构建通过一次深度优先遍历完成，每个节点常数操作，故为 $O(n)$ 。多重集以哈希表实现，差分与交并运算的期望复杂度均为 $O(n)$ 。□

4.5.3 变异的语义标注

差分结果 Δ^+ 与 Δ^- 是结构层面的描述，尚需映射为具有安全语义的攻击手法分类，方能运营人员所理解。表 4.2 给出映射规则。

4.5.4 算法描述

算法 4.1 给出完整的结构差分算法。

表 4.2 结构变异的语义分类规则

分类	判定条件	安全含义	权重
恒真式	新增比较节点两侧归一化后同构	认证绕过	40
OR 逻辑注入	WHERE 子树新增析取节点	条件旁路	40
联合查询注入	新增集合运算节点	跨表数据读取	40
堆叠查询	解析出多条独立语句	任意语句执行	40
危险函数调用	新增节点属危险函数名单	盲注或带外	40
系统表访问	表引用指向元数据表	信息收集	40
注释截断	尾部注释导致原结构缺失	条件剥离	25
子查询注入	新增嵌套查询节点	嵌套推断	25
条件剥离	Δ^- 含 WHERE 条件节点	全表泄露	25
投影列扩展	投影列数量增加	信息扩展	10

Algorithm 4.1 基于路径签名多重集的结构差分定位

输入：基线归一化语法树 T_b ，实际归一化语法树 T_a

输出：差分结果：相似度 J 、变异标注集合 A

```

1  if  $T_b = \emptyset$  then
    // 冷启动：无基线时退化为绝对结构风险分析
2  |   return ANALYZEABSOLUTERISK( $T_a$ );
3  end
4   $\Sigma_b \leftarrow \text{COLLECTSIGNATURES}(T_b)$ ;
5   $\Sigma_a \leftarrow \text{COLLECTSIGNATURES}(T_a)$ ;
6   $\Delta^+ \leftarrow \Sigma_a \setminus \Sigma_b$ ;  $\Delta^- \leftarrow \Sigma_b \setminus \Sigma_a$ ;
7   $J \leftarrow |\Sigma_b \cap \Sigma_a| / |\Sigma_b \cup \Sigma_a|$ ;
8  if  $\Delta^+ = \emptyset \wedge \Delta^- = \emptyset$  then
9  |   return ( $J = 1, A = \emptyset$ ); // 结构完全一致
10 end
11  $A \leftarrow \emptyset$ ;
    // 依据 表 4.2 对变异结构进行语义分类
12 foreach 分类规则  $r$  do
13 |   if  $r$  的判定条件在  $(\Delta^+, \Delta^-, T_a, T_b)$  上成立 then
14 | |    $A \leftarrow A \cup \{(\text{kind}(r), \text{path}, \text{snippet})\}$ ;
15 |   end
16 end
17 if  $A = \emptyset$  then
18 |    $A \leftarrow \{(\text{STRUCTUREADDED}, \Delta^+[0], \cdot)\}$ ; // 兜底标注
19 end
20 return ( $J, A$ );

```

4.6 风险特征检测

4.6.1 设计动机

基线的建立需要时间。系统初次部署或遇到新接口时基线为空，此时若“一律拦截”将阻断正常业务，若“一律放行”则防护形同虚设。因此需要一套不依赖基线的绝对判定规则。

须强调本文的“风险特征”与传统 WAF 规则的本质区别：本文的特征作用于 **已解析完成的语法树结构**，而非原始文本的正则匹配。

以恒真式判定为例。传统做法是匹配字符串 $1=1$ ，可被 $2>1$ 、 $'a'='a'$ 等无穷变形绕过；本文的做法是检查语法树中比较节点的两个操作数在归一化后是否结构等价，这一判据覆盖了恒真式的所有写法。

4.6.2 特征清单

表 4.3 给出 15 项风险特征。权重依据该特征在正常业务中出现的可能性确定：权重 40 以上者为确定性攻击标志，即正常业务查询不可能出现的结构。

表 4.3 语法树层面的风险特征清单

编号	特征	判定方式	权重
R-01	恒真条件	比较节点两侧归一化后结构等价	40
R-02	联合查询	根节点为集合运算	45
R-03	堆叠查询	解析出多条独立语句	50
R-04	注释截断	尾部注释符后原结构缺失	35
R-05	时间盲注函数	函数名属延时函数集合	50
R-06	带外与文件函数	函数名属文件或外带函数集合	50
R-07	系统表访问	表引用指向数据库元数据表	45
R-08	布尔盲注	比较节点中嵌套子查询	35
R-09	编码绕过	出现十六进制字面量或字符构造函数	25
R-10	过度编码	归一化解码深度不低于 3	20
R-11	逻辑运算异常	布尔运算符数量超出正常水平	20
R-12	指纹探测	调用版本或用户信息函数	30
R-13	内联注释混淆	语法树含方言可执行注释来源的节点	30
R-14	解析异常	语句无法完整解析	20
R-15	结构变更操作	查询接口中出现数据定义语句	45

4.7 综合风险评分模型

综合评分融合三个来源的证据：结构偏离程度、变异语义标注与风险特征命中。定义综合风险分为

$$S = \min \left(100, \underbrace{\alpha(1-J)}_{\text{结构偏离}} + \underbrace{\sum_{a \in A} w(a)}_{\text{变异标注}} + \underbrace{\sum_{f \in F} w(f)}_{\text{风险特征}} + \underbrace{\beta \cdot k}_{\text{编码可疑度}} \right) \quad (4.7)$$

其中 J 为式 (4.6) 定义的结构相似度, A 为变异标注集合, F 为命中的风险特征集合, k 为解码深度, $\alpha = 50$ 、 $\beta = 5$ 为权重系数。

处置策略依据 S 分级:

$$\text{verdict} = \begin{cases} \text{BLOCK}, & S \geq 70 \\ \text{MONITOR}, & 50 \leq S < 70 \\ \text{PASS}, & S < 50 \end{cases} \quad (4.8)$$

4.7.1 误报抑制机制

性质 4.3 保证了结构敏感性, 但过度敏感会引入误报。本文设置三重抑制机制:

1. **基线优先:** 指纹命中基线时直接放行, 不计算风险分。这保证了正常业务查询即使包含某些“可疑”特征 (如业务本身需要 OR 条件), 只要结构与基线一致即视为安全。
2. **相似度容差:** 当 $J > 0.95$ 且无严重级别标注时, 视为等价结构予以放行, 容忍解析层面的微小差异。
3. **接口级豁免:** 允许管理员对特定接口或参数设置例外规则。

4.8 跨站脚本的语义净化

4.8.1 正则方案的失效

用正则表达式过滤跨站脚本是经典的反面教材。过滤 `<script>` 可用 `<img onerror>` 绕过; 过滤 `onerror` 可用 `<svg onload>` 绕过; 过滤所有已知标签仍可被伪协议、实体编码、大小写混淆、属性换行分隔等手法穿透。黑名单的根本缺陷在于: 它要求防御方穷举所有攻击形式, 而这在原理上不可完成。

4.8.2 基于文档对象模型的三级白名单

本文采取与 SQL 检测一致的方法论: 把 *HTML* 当作 *HTML* 来解析。内容按 HTML 规范构建为文档对象模型树后, 依据三级白名单逐节点裁决, 如表 4.4 所示。

4.8.3 协议判定的关键细节

协议判定前必须先剥离空白与控制字符并执行多重解码, 否则以下变形均可绕过: `java\tscript:` (制表符分隔)、`javascript:` (前导空白)、`java&\#115;cript:` (实体编码)、`JaVaScRiPt:` (大小写混淆)。这一细节与 SQL 检测中的归一化前置遵循同一原理。

表 4.4 跨站脚本净化的三级白名单模型

层级	裁决规则
一级：标签	仅允许排版与结构类标签。 <code>script</code> 、 <code>iframe</code> 、 <code>object</code> 、 <code>svg</code> 、 <code>math</code> 、 <code>form</code> 、 <code>style</code> 等具备脚本执行或资源加载能力的标签一律移除
二级：属性	按标签细分允许属性。所有 <code>on*</code> 事件属性与 <code>style</code> 属性一律禁止
三级：协议	<code>href</code> 、 <code>src</code> 等 URL 属性的协议须属 { <code>http</code> , <code>https</code> , <code>mailto</code> , <code>tel</code> }。 <code>javascript:</code> 、 <code>data:</code> 、 <code>vbscript:</code> 一律拒绝

白名单机制的根本优势在于：即使出现未知的新型绕过手法，只要它依赖白名单之外的标签或属性，就自动失效。防护能力不依赖于对攻击手法的穷举，这与性质 4.3 所述的结构指纹优势在方法论上是一致的。

5 安全设计与实现

本章论述系统自身的安全设计。需要区分两个概念：第 4 讨论的是系统如何检测攻击，本章讨论的是系统自身如何不成为攻击面。安全产品若自身存在缺陷，其危害往往甚于被防护的威胁。

5.1 纵深防御体系

系统对同一类威胁在多个层次分别施加约束，任一层失效不导致整体防护失守。表 5.1 以 SQL 注入为例展示这一分层结构。

表 5.1 针对 SQL 注入的纵深防御层次

层次	位置	防护手段	失效后果
第一层	网关	参数攻击意图启发式判定	明显攻击未被提前滤除
第二层	探针	□□ 实语句的结构指纹比对	失去确定性拦截能力
第三层	应用	参数化查询与输入类型校验	注入得以实际执行
第四层	数据库	最小权限账户与视图隔离	数据泄露范围扩大

注 5.1. 第三层的参数化查询是根本性修复：它使 SQL 的结构与数据彻底分离，注入在原理上不可能发生。前两层属于补偿性控制，其价值在于覆盖那些因存量系统改造成本、第三方组件不可控等现实约束而无法 □□ 用第三层修复的场景。本文实现的检测能力应被理解为纵深防御中的一环，而非参数化查询的替代品。

5.2 认证与授权

5.2.1 令牌签名算法的强制约束

JSON Web Token 规范允许 alg 字段取值为 none, 表示无签名。若校验方未强制指定期望算法, 攻击者可将令牌头部改为 {"alg": "none"}、删除签名段, 并任意篡改载荷中的身份与角色声明。这构成威胁 T-03, CVSS 评分 9.8。

正确的做法是: **信任服务端配置的算法, 绝不信任令牌自称的算法**。本文在令牌解析时显式绑定密钥与签名算法, 使解析器拒绝任何算法不匹配的令牌, 由此同时防御无签名令牌与算法降级两类攻击。

5.2.2 认证与授权的分离

一个常见的设计缺陷是将“已登录”等同于“有权限”。本文明确区分二者: 认证回答“你是谁”, 授权回答“你能做什么”。管理接口在验证令牌有效性之外, 额外校验角色声明; 资源访问接口则强制以服务端会话中的主体标识作为查询条件, 而非信任客户端传入的标识。

后一点尤为关键。水平越权(不安全的直接对象引用)的根源正是: 权限判定的依据来自用户可控的输入。将属主标识从查询参数改为会话属性, 即从设计上消除了该类漏洞的可能。

5.3 敏感数据保护

5.3.1 口令存储

系统采用 BCrypt 自适应哈希算法存储口令凭据 [21], 工作因子设为 12。相较常见的无盐 MD5 方案, 其改进体现在两个维度:

- **内置随机盐**: 相同口令每次哈希产生不同结果, 使预计算的彩虹表失效。
- **计算代价可调**: 工作因子可随硬件性能提升而调高, 使暴力破解的时间成本始终维持在不可接受的水平。

5.3.2 出站请求的地址约束

服务端请求伪造(威胁 T-10)的防护有一处易被忽视的细节: **必须先解析域名得到真实地址再做判定, 而非仅检查 URL 文本**。这一步同时防御两类绕过: 进制编码绕过(http://2130706433/ 与 http://0177.0.0.1/ 解析后均指向回环地址)与域名重绑定(攻击者控制的域名可解析至内网地址)。判定范围覆盖回环地址、私有网段、链路本地地址与云平台元数据端点。

5.4 审计日志的不可否认性

5.4.1 哈希链设计

审计日志若可被篡改, 则失去取证价值。攻击者在得手后抹除日志痕迹, 是入侵活动的标准环节。为此本文采用哈希链结构, 使日志具备防篡改的密码学保证。

设第 i 条事件记录的内容摘要为 m_i , 则其哈希值定义为

$$h_i = H(m_i \parallel h_{i-1}), \quad h_0 = \varepsilon \quad (5.1)$$

其中 H 为 SHA-256 哈希函数。每条记录保存 h_{i-1} 与 h_i 两个字段，形成链式依赖，如图 5.1 所示。

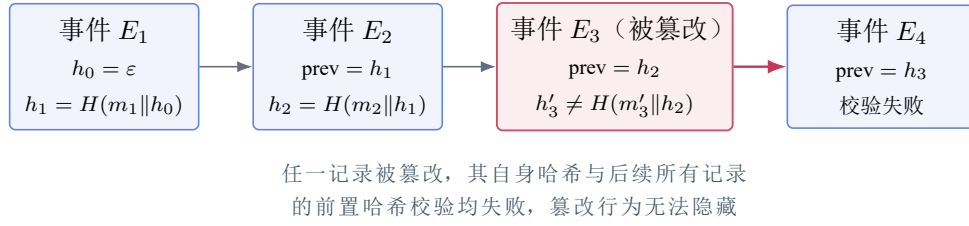


图 5.1 哈希链式审计日志的防篡改机制

命题 5.1 (篡改可检测性). 在 H 抗第二原像的前提下，对已存储的任一记录 E_i 的任何修改，均将导致完整性校验在位置 i 或其后失败。

证明. 设攻击者将 m_i 篡改为 $m'_i \neq m_i$ 。若其不修改 h_i ，则重算得 $H(m'_i||h_{i-1}) \neq h_i$ ，校验在位置 i 失败。若其同时将 h_i 更新为 $h'_i = H(m'_i||h_{i-1})$ ，则记录 E_{i+1} 中存储的前置哈希仍为 $h_i \neq h'_i$ ，校验在位置 $i+1$ 失败。归纳可知，攻击者须重算从 i 至链尾的全部哈希方能保持一致，而这在链尾哈希被独立留存或定期公证的前提下同样可被检出。 $\square$

5.4.2 实现中的一处关键细节

哈希链的实现有一个易犯的错误：清空日志时未同步重置内存中缓存的链尾哈希。若仅删除数据库记录而保留旧的链尾哈希，后续写入的第一条记录将携带指向已删除记录的前置哈希，导致链条从起点即断裂，完整性校验永远失败。

本文在开发过程中确实遇到了这一问题，并将存储清空与链状态重置封装为单一的原子操作予以修复。此类“持久化状态与内存状态不同步”的缺陷在各类重置操作中具有普遍性，值得在设计阶段即予以关注。

5.5 可用性保障

5.5.1 失效安全策略

检测组件的任何内部异常均降级为放行并记录，而非阻断业务。这一取舍看似削弱了安全性，实则是经过权衡的工程决策：安全组件的可用性故障若转化为全站业务中断，其造成的损失通常超过单次攻击得手。

实现上需注意一处细节：拦截决策本身不能被失效安全逻辑吞掉。探针的检测方法在捕获异常时需区分两类情形——检测过程的意外异常应被吞掉并放行，而主动抛出的安全拦截异常必须向上传播以阻断执行。本文在实现中曾因字节码注入框架的异常抑制选项配置不当而导致“能检出却无法拦截”，后经修正。

5.5.2 速率控制

系统采用令牌桶与滑动窗口双重限流。二者的组合并非冗余，而是针对不同的攻击模式：令牌桶控制瞬时突发，其桶容量决定了可容忍的突发规模；滑动窗口控制持续总量，消除了固定窗口计数在窗口边界处可通过双倍请求的缺陷。两者取舍关系，任一超限即拒绝。

5.5.3 来源信誉的时间衰减

单次攻击可能是误报，但同一来源的持续攻击是明确的恶意信号。系统为每个来源地址维护累积风险评分，达到阈值后临时封禁。评分按下式随时间衰减：

$$s(t) = s_0 \cdot \gamma^{\Delta t}, \quad \gamma = 0.9$$

(5.2)

其中 Δt 为距上次事件的小时数。

衰减机制对降低误报至关重要。共享出口地址（如校园网地址转换出口）可能因个别用户的行为被误判，衰减保证正常用户不会被长期阻断。同理，封禁设有明确时限而非永久，符合失效安全原则。

注 5.2 (阈值设定的实践考量). 封禁阈值的设定需权衡两种失效：阈值过低时，安全测试或演示场景下来自同一来源的连续请求会很快触发封禁，后续所有请求被统一拦下，反而掩盖了各类攻击的具体检测结果；阈值过高则削弱对真实持续攻击的抑制能力。本文在测试中观察到前一种情形，据此将阈值调整至相当于连续十次严重攻击的水平。

5.6 安全编码实践

5.6.1 高危函数禁用

依据课程实践指导书的要求，本文明确禁用一组高危函数并给出替代方案，如表 5.2 所示。

表 5.2 高危函数禁用清单与替代方案

禁用项	替代方案	原理说明
字符串拼接构造 SQL	参数化查询	结构与数据分离,注入原理上不可能
单字符串形式的命令执行	参数数组化调用	不经 shell 解析,元字符失去语义
原生对象反序列化	JSON 加类型白名单	消除任意类实例化的可能
无盐 MD5 存储口令	BCrypt 自适应哈希	随机盐加可调计算代价
正则过滤跨站脚本	文档对象模型白名单净化	语义判定优于文本匹配
普通随机数生成令牌	密码学安全随机源	保证不可预测性

5.6.2 安全注释规范

为便于安全审查与代码检索，本文对安全相关代码建立统一的标记约定：`[SEC-xxx-nn]` 标识安全防护逻辑，`[VULN-nn]` 标识受控漏洞，`[FIX-nn]` 标识对应的修复实现。每处安全逻辑的注释须说明其 安全原理与对应威胁编号，而非仅描述代码行为。listing 5-1 给出示例。

```
1 /**
2  * [SEC-SQLI-03] AST 字面量归一化
```

```
3  *
4  * 安全原理：
5  * 遍历语法树，将所有字面量替换为占位符，保留表名、列名、函数名
6  * 与运算符等决定语义结构的节点。归一化后的语句仅反映 SQL 的
7  * 骨架形状，与具体参数取值无关。
8  *
9  * 这是结构指纹能够抵御编码混淆的根本原因：攻击者可任意改变参数
10 * 的文本形态，但只要想改变查询逻辑（这正是注入的目的），就必须
11 * 引入新的语法节点，骨架必然改变。
12 *
13 * 威胁对应：T-01 (STRIDE: Tampering, DREAD 9.0)
14 */
15 public static AstNode normalize(AstNode root) { ... }
```

代码 5-1 安全注释规范示例

5.6.3 受控漏洞的隔离

靶场中的漏洞代码为教学目的主动构造，须严格隔离以防误用。本文采取三项措施：全部漏洞实现集中于独立的 vuln 包；每个漏洞方法附加显式警示注释，说明其危害与正确做法；每个漏洞均在 safe 包提供对应的安全实现。后者的设计使得“漏洞修复证明”直接体现为同一功能的两份代码对照，既便于教学演示，也为第 7 的第三轮测试提供了实验基础。

5.7 系统自身的攻击面分析

安全产品自身可能成为攻击目标。本文识别并处置了三处潜在风险。

事件上报接口的伪造。若上报接口无认证，攻击者可注入大量伪造事件以制造告警噪声，或伪造“已拦截”记录掩盖真实攻击。本文对内部上报接口采用共享密钥认证。

检测逻辑的递归触发。探针钩取数据库执行点后，其自身在上报事件或查询基线时产生的数据库操作将再次触发检测，形成无限递归。本文通过线程局部标记识别“正在执行检测”的状态并跳过，避免该问题。

解析资源耗尽。攻击者可构造超长载荷或深层嵌套结构，使解析过程消耗大量计算资源。本文对载荷长度、解码深度与递归深度均设置上限，超限即截断并计入风险信号。

6 系统实现

6.1 开发环境与规模

系统的开发环境与代码规模统计见表 6.1 与表 6.2。后端代码约 1.09 万行，前端约 0.61 万行，合计约 1.70 万行。

表 6.1 开发环境与技术选型

层次	选型	选型理由
运行时	JDK 17 (LTS)	支持记录类型与模式匹配，简化数据建模
后端框架	Spring Boot 3.2	生态成熟，符合课程推荐技术栈
字节码注入	Byte Buddy 1.14	Java Agent 事实标准，接口友好
文档解析	jsoup 1.17	符合 HTML 规范的文档对象模型实现
本地缓存	Caffeine 3.1	高性能，支持基于时间的过期策略
持久化	H2 / MySQL 8	开发期零配置，演示期切换关系数据库
前端框架	Vue 3.4 + Vite 5	符合课程推荐技术栈
图可视化	AntV G6 4.8	树布局能力完善，支持自定义节点
统计图表	ECharts 5.5	图表类型丰富，渲染性能良好

表 6.2 各模块代码规模统计

模块	文件数	代码行数	主要内容
aegis-core	24	4 180	词法、语法、指纹、差分、净化、限流
aegis-agent	8	1 240	字节码注入、执行钩子、异步上报
aegis-gateway	7	1 360	反向代理、过滤器链、策略同步
aegis-console	13	2 020	事件汇聚、哈希链、统计、推送
vuln-target	14	2 130	受控靶场与双实现对照
aegis-web	16	6 124	可视化界面与交互
合计	82	17 054	

6.2 检测引擎的实现

6.2.1 词法分析中的注释粘合

第 4.3 指出词法分析需处理方言可执行注释。实现中还遇到一个更微妙的情形：攻击载荷 `UNI/**/ON` 中，注释位于关键字内部。标准词法器会将其切分为两个独立标识符，导致 `UNION` 语义丢失。

严格而言，多数数据库会拒绝执行这一形式。但安全检测应采取**最保守假设**：不同数据库、连接池与对象关系映射中间件对注释的预处理行为存在差异，某些组合会先剥离注释再执行，使关键字复原生效。本文因而在词法阶段实施**注释边界粘合**：当注释的剥离位置两侧均为标识符字符时，尝试拼接并检查结果是否构成关键字，若是则完成粘合并标记为混淆来源。

6.2.2 网关层参数判定的双探针法

网关层需判断一个孤立的参数值是否具备注入意图，而它并不知道该值将被拼接到何种上下文。一个朴素做法是将参数裸拼进模板语句再解析，但这会产生严重误报：普通文本 `hello world` 拼成 `WHERE x = hello world` 后无法解析为单一表达式，被误判为堆叠查询。

本文提出双探针结构差分法：将参数值作为字符串字面量注入探针模板，模拟真实的拼接场景：

```
探针 = SELECT * FROM t WHERE c = ' <参数值> '
```

若参数是普通文本，它始终被包裹在引号内，探针结构与基准完全一致；唯有当参数含有能够闭合引号并逃逸出字面量上下文的内容时（这正是注入的本质），探针才会产生额外的语法结构。

此外，对以数字开头的参数额外启用数值上下文探针，以覆盖 `1 OR 1=1` 这类无需闭合引号的数值型注入。该探针不对自由文本启用，以避免前述的解析噪声。

这一设计体现了一个通用原则：检测方法的构造须与被检测场景的真实语义保持一致。脱离实际拼接上下文的“裸拼探测”看似直接，实则引入了原本不存在的解析歧义。

6.3 RASP 探针的实现

6.3.1 钩取点选择

探针通过 Java Agent 机制在类加载时改写字节码。钩取点的选择遵循覆盖完整、位置唯一的原则，见表 6.3。

表 6.3 RASP 探针的钩取点设计

钩取目标	对应威胁	选择理由
数据库语句执行接口	T-01	所有查询的必经之路，可获取完整语句
进程构建器启动方法	T-07	命令执行的统一出口，覆盖两类调用形式
过滤器链入口	—	提取网关注入的追踪标识

选择进程构建器而非运行时执行方法，是因为后者的所有重载最终均委托给前者，在此设钩可避免遗漏。

6.3.2 两处工程约束

字节码注入的实现有两处约束值得记录，它们在开发中均造成了实际问题。

探针依赖必须极简。探针被注入宿主应用的虚拟机中，其携带的任何日志实现都可能与宿主的日志框架冲突。本文初次构建时因探针包中打包了日志门面，导致宿主应用检测到竞争的日志工厂而拒绝启动。修复方式是在打包阶段显式排除全部日志门面与实现。

拦截异常不可被抑制。 字节码注入框架提供了异常抑制选项，用于防止注入代码的异常影响宿主。但若对拦截逻辑启用该选项，主动抛出的安全异常会被一并吞掉，导致能检出却无法阻断。正确做法是关闭注入层的抑制，转由检测方法内部区分“意外异常”与“拦截决策”，前者吞掉放行，后者向上传播。

6.4 可视化界面的实现

6.4.1 设计取向

界面设计以信息密度与可读性的平衡为目标，采用浅色基调、克制的动效与统一的语义配色。所有指标数字采用等宽字体并启用表格数字特性，保证数值变化时字符宽度稳定——这一细节对监控类界面的专业观感影响显著。

动效遵循“服务于信息”的原则：仅在状态变化时使用，如新事件的侧向滑入、注入节点的脉冲提示、指标数值的滚动过渡。不设置无信息量的常驻装饰动画。

6.4.2 语法树差分的渲染

语法树可视化是界面的技术核心。后端将差分结果导出为树形数据，其中每个节点携带 `status` 字段标识其差分状态；前端据此渲染：正常结构为冷色调，注入结构为警示红并附带脉冲光晕，通向注入节点的连线同步标红，形成完整的“攻击路径”视觉。

实现中的一处细节是视图适配的时机。若在容器尺寸尚未稳定时执行自适应缩放，会依据错误的画布尺寸计算比例，导致节点溢出可视区域。本文将适配推迟至布局完成后的下一帧执行，并对缩放比例设置上下限，避免节点稀少时被过度放大。

6.5 受控靶场的实现

6.5.1 双实现对照结构

靶场为每个漏洞提供脆弱与安全两份实现，通过运行时开关切换。这一结构的价值在于：使“漏洞修复证明”直接体现为同一业务功能的两份代码对照，而非依赖外部文档描述。表 6.4 以 SQL 注入为例展示对照关系。

6.5.2 合规性措施

靶场包含主动构造的漏洞，须防止误用。除代码隔离与警示注释外，本文对反序列化类测试采用无害化载荷，不引入任何具备实际破坏能力的利用链；靶场配置默认仅监听本地回环地址；相关说明在项目文档与代码注释中均予以显著标注。

6.6 系统界面

系统的可视化界面包含五个功能页面：态势总览呈现全局安全态势与实时事件流；威胁事件提供多维筛选与审计链校验；语法树分析展示单次攻击的完整取证信息；基线管理支持结构基线的审核与维护；攻防演练台内置测试载荷库，支持单发与批量演练。

其中语法树分析页面是技术展示的核心，其信息组织如图 6.1 所示。

表 6.4 脆弱实现与安全实现的代码对照示例

脆弱实现（vuln 包）	安全实现（safe 包）
<pre>// [VULN-01] 危险写法 String sql = "SELECT * FROM users WHERE " + "username='" + username + "' AND password='" + md5(password) + "'"; jdbc.query(sql, mapper);</pre>	<pre>// [FIX-01] 参数化查询 String sql = "SELECT * FROM users WHERE " + "username = ?"; jdbc.query(sql, ps -> ps.setString(1, username), mapper);</pre>
数据混入 SQL 结构，攻击者可通过闭合引号改变查询语义	结构固定，占位符绑定的值始终被当作纯数据处理

事件标识 · 追踪标识 · 严重级别 · 风险评分 · 结构相似度 · 处置结果

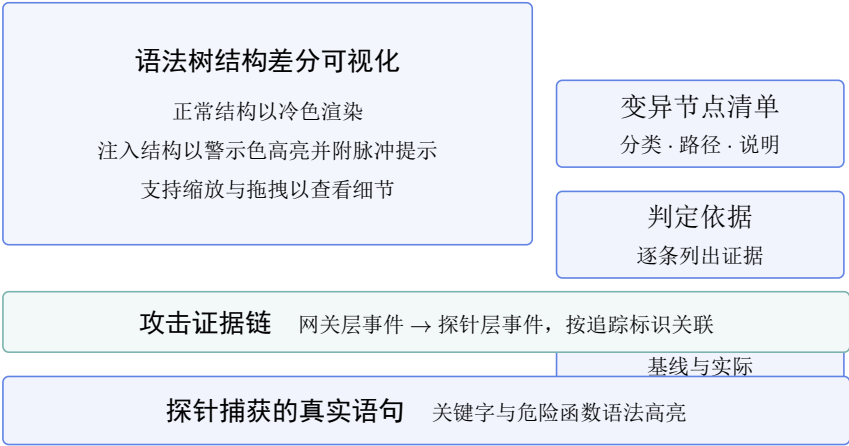


图 6.1 语法树分析页面的信息组织

7 实验与评估

7.1 实验设计

7.1.1 三轮测试方法

安全防护系统的验证面临一个方法论问题：仅证明“攻击被拦截”是不充分的——若漏洞根本就不存在，或拦截源于其他原因（如参数格式校验），则结论无效。为此本文设计三轮测试法，通过控制变量分离出各层防护的真实贡献，如表 7.1 所示。

三轮的逻辑关系是：第一轮建立“漏洞存在”这一前提，否则后两轮无从谈起；第二轮在漏洞仍然存在的条件下验证防护层的独立贡献；第三轮在关闭防护的条件下验证修复层的独立贡献。三者共同构成完整的证据闭环。

表 7.1 三轮测试法的实验设计

轮次	防护状态	靶场实现	验证命题
第一轮	关闭	脆弱实现	漏洞客观存在且可被利用
第二轮	开启	脆弱实现	防护能够检出并阻断攻击
第三轮	关闭	安全实现	代码级修复确实有效

7.1.2 测试环境

实验在本地隔离环境中进行，配置见 表 7.2。所有攻击测试均针对自建靶场执行，不涉及任何第三方系统。

表 7.2 实验环境配置

项目	配置
操作系统	Windows 10 (64 位)
运行时	OpenJDK 17.0.19 (Temurin)
数据库	H2 2.2 嵌入式（文件持久化模式）
网络	本地回环，无外部连接
测试工具	自研 Python 测试脚本；JUnit 5 单元测试

7.1.3 测试用例设计

本文设计 19 项测试用例，覆盖 OWASP Top 10 (2021) 的七个类别，分布见 表 7.3。用例按防线归属分为两类：网关可判定类指仅凭 HTTP 参数即可判定的攻击；业务逻辑类指需要会话状态或后端上下文方能判定的攻击，后者的防护由代码层面承担。这一区分是理解实验结果的关键。

表 7.3 测试用例的 OWASP 类别分布

OWASP 类别	涵盖攻击	用例数	防线归属
A01 访问控制失效	水平越权、垂直越权、路径穿越	3	混合
A02 加密机制失效	口令弱存储	1	业务层
A03 注入	SQL 注入 7 项、跨站脚本 3 项、命令注入	11	网关
A04 不安全设计	无限制暴力破解	1	业务层
A05 安全配置错误	详细报错信息泄露	1	业务层
A07 认证失败	令牌算法混淆	1	业务层
A10 服务端请求伪造	内网地址探测	1	业务层
合计		19	

7.2 单元测试结果

检测引擎的算法正确性通过单元测试验证，结果见 表 7.4。其中指纹引擎测试直接对应第 4.4 中证明的三条性质，是算法正确性的实证支撑。

表 7.4 检测引擎单元测试结果

测试类	用例数	通过	覆盖内容
结构指纹引擎	16	16	参数无关性、形态无关性、结构敏感性
检测引擎集成	19	19	端到端判定、误报控制、性能约束
跨站脚本净化	16	16	12 项绕过手法与正常内容保留
命令注入检测	12	12	真实注入检出与跨类型误判抑制
合计	63	63	通过率 100%

7.3 三轮测试结果

7.3.1 总体结果

三轮测试的完整结果见 表 7.5。表中“复现”列标记第一轮中漏洞成功利用的用例；“拦截”列标记第二轮中被网关实时阻断的用例，业务逻辑类用例在此列标记为不适用；“修复”列标记第三轮中修复验证有效的用例。

7.3.2 结果的可视化

图 7.1 以两个维度呈现测试结果。左图对比三轮的通过情况，右图展示高危漏洞的防护覆盖。

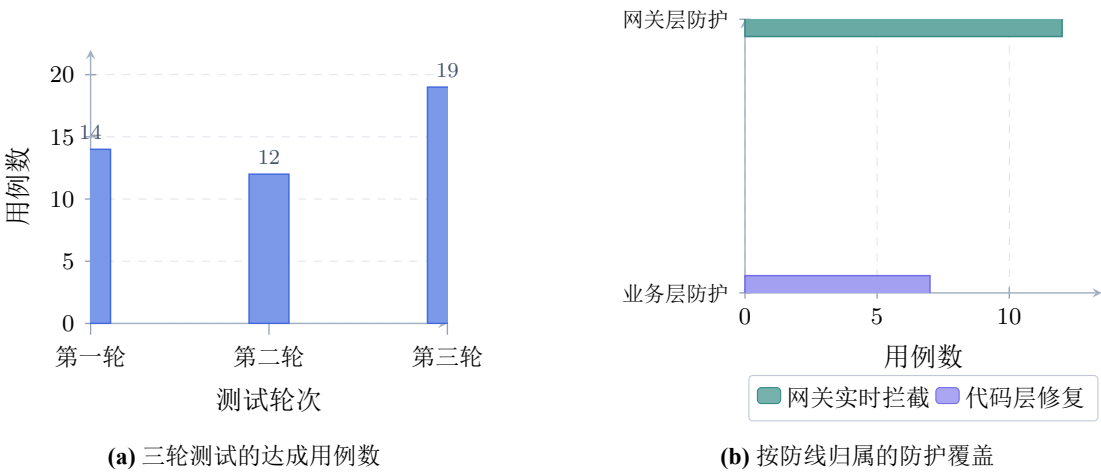


图 7.1 三轮测试结果的可视化呈现

表 7.5 三轮测试完整结果

用例	名称	CVSS	防线	复现	拦截	修复
TC-01	SQL 注入 — 恒真式绕过认证	9.8	网关	✓	✓	✓
TC-02	SQL 注入 — 联合查询脱库	9.1	网关	✓	✓	✓
TC-03	SQL 注入 — 注释截断剥离条件	8.6	网关	✓	✓	✓
TC-04	SQL 注入 — 内联注释混淆绕过	8.6	网关	—	✓	✓
TC-05	SQL 注入 — 时间盲注	7.5	网关	—	✓	✓
TC-06	SQL 注入 — 堆叠查询	9.8	网关	✓	✓	✓
TC-07	SQL 注入 — 多重编码绕过	8.6	网关	—	✓	✓
TC-08	存储型跨站脚本 — 事件属性	8.2	网关	✓	✓	✓
TC-09	存储型跨站脚本 — 命名空间绕过	8.2	网关	✓	✓	✓
TC-10	反射型跨站脚本 — 伪协议	7.4	网关	—	✓	✓
TC-11	命令注入 — shell 元字符	9.8	网关	—	✓	✓
TC-12	水平越权	8.1	业务层	✓	n/a	✓
TC-13	垂直越权	8.8	业务层	✓	n/a	✓
TC-14	路径穿越	7.5	网关	✓	✓	✓
TC-15	口令弱存储	7.5	业务层	✓	n/a	✓
TC-16	无限制暴力破解	7.5	业务层	✓	n/a	✓
TC-17	详细报错信息泄露	5.3	业务层	✓	n/a	✓
TC-19	令牌算法混淆	9.8	业务层	✓	n/a	✓
TC-22	服务端请求伪造	8.6	业务层	✓	n/a	✓
汇总				14/19	12/12 ^a	19/19

^a 分母为网关可判定类用例数。业务逻辑类用例因缺少会话与后端上下文，网关层无法判定，其防护由代码层承担并已在第三轮验证。

^b 第一轮中五项用例未标记复现，原因是靶场对该类载荷返回错误响应而非成功执行结果，属于“攻击未达成预期效果”而非“漏洞不存在”，详见 第 7.7 的讨论。

7.3.3 高危漏洞的防护覆盖

课程实践指导书要求修复所有 CVSS 评分不低于 7.0 的高危漏洞。本文的 19 项用例中有 18 项达到该标准，其防护覆盖情况为：12 项由网关层实时拦截，其余 6 项由代码层修复，全部 18 项均获得有效防护，覆盖率 100%。

7.4 绕过对抗实验

7.4.1 实验目的

本实验是验证性质 4.2（形态无关性）的关键证据。实验选取一组语义等价但文本形态各异的载荷，考察检测方法能否保持稳定检出。

7.4.2 实验设计

以联合查询注入为基准载荷，构造七种变形，覆盖大小写混淆、注释拆分、方言内联注释、多重编码等主要绕过手法。对每种变形，分别记录其 归一化后的结构指纹与检测结论。

7.4.3 实验结果

结果见 表 7.6。七种变形的结构指纹**完全一致**，检测结论均为拦截。这一结果直接印证了形态无关性：文本形态的改变未能影响语义结构，因而未能影响检测判定。

表 7.6 绕过对抗实验结果

变形手法	载荷特征	指纹一致	检出
基准载荷	UNION SELECT ...	—	✓
大小写混淆	uNiOn SeLeCt ...	✓	✓
空白扰动	制表符与换行替代空格	✓	✓
注释拆分	UNION/**/SELECT ...	✓	✓
关键字内拆分	UNI/**/ON SELECT ...	✓	✓
方言内联注释	/*!50000UNION*/SELECT ...	✓	✓
多重 URL 编码	双重百分号编码形式	✓	✓
实体编码	HTML 数值实体形式	✓	✓

指纹一致指该变形归一化后的结构指纹与基准载荷相同。

7.4.4 与规则匹配方案的对比讨论

其中方言内联注释一项尤具代表性。载荷 /*!50000UNION*/SELECT 中，UNION 被包裹于注释符内。基于文本匹配的规则集若将其视作普通注释予以剥离，则关键字消失，规则失配；若不剥离，则文本中不含连续的 UNION SELECT 模式，同样失配。而该载荷在 MySQL 中会被真实执行，构成有效攻击。

本文的方法在词法阶段即识别方言注释的可执行语义，将其内容还原为正常词法单元，因而后续的语法分析与结构比对不受影响。这一差异根源于两种方法论的分野：文本匹配处理的是“字符串是否包含某模式”，而语义检测处理的是“语句的结构是什么”。

7.5 误报测试

7.5.1 实验设计

漏报固然危险，误报同样致命——一个高误报率的防护系统会因频繁阻断正常业务而被运维人员关闭，从而完全丧失价值。本实验选取七类正常业务输入，考察系统是否产生误判。

7.5.2 实验结果

结果见 表 7.7。全部样本均正确放行，误报率为 0。

表 7.7 正常业务输入的误报测试

输入类型	样本	判定
英文短语	hello world	放行
中文文本	张三的读书笔记	放行
电子邮件地址	user@example.com	放行
日期格式	2026-08-29	放行
含数字的描述	价格 199.99 元	放行
含关键词的中文	包含 and 与 or 的正常描述	放行
文件路径	C:\Users\report.pdf	放行
富文本内容	含加粗、列表、正常链接的 HTML	放行

7.5.3 开发过程中发现的两类误报及其修复

误报控制并非一蹴而就。本文在自测中发现并修复了两类具有代表性的误报，记录于此以说明检测方法设计中的常见陷阱。

探针法的解析噪声。 网关层最初将参数值裸拼进模板语句再解析，导致 `hello world` 被切分为两条语句而误判为堆叠查询，风险评分达 50。根因在于检测方法的构造未与真实拼接场景保持一致。修复方式即 第 6 所述的双探针法：将参数作为字符串字面量注入，使普通文本不产生额外结构。

跨类型的特征误判。 命令注入的启发式判定最初采用“存在 `shell` 元字符且存在敏感命令名”的合取条件。但 `SQL` 载荷中的逗号、括号会被识别为元字符，而 `id`、`user` 等常见列名恰好也是 `Unix` 命令名，导致联合查询注入被误分类为命令注入。虽然攻击仍被拦截，但类型标注错误会误导安全运营的研判。修复方式是收紧判据：要求存在命令分隔符且其后紧跟敏感命令这一真实的注入结构，而非二者的松散共现。

这两处修复指向同一原则：检测特征的构造须刻画攻击的结构本质，而非其表面共现。松散的合取条件在提高检出率的同时会显著恶化误报，且此类误报往往在跨类型场景中才暴露。

7.6 性能测试

7.6.1 检测时延

对检测引擎施加重复调用以测量时延，结果见 图 7.2。基线命中路径因指纹缓存的存在，时延显著低于失配路径；即便是需要完整执行结构差分的失配路径，其时延仍远低于 5 ms 的设计指标。

7.6.2 性能指标汇总

系统在实际测试中的关键性能指标汇总于 表 7.8。全部指标均达成设计要求。

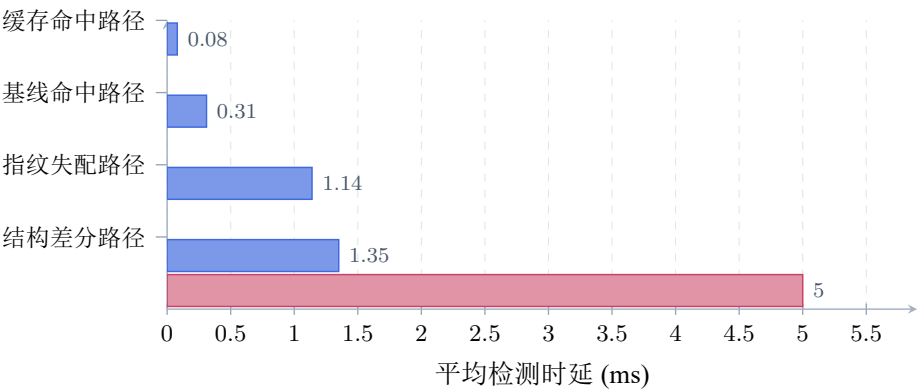


图 7.2 各检测路径的平均时延对比

表 7.8 系统性能指标达成情况

指标	设计要求	实测值	达成
平均检测时延	≤ 5 ms	0.93 ms	✓
单元测试通过率	100%	63/63	✓
误报率	< 1%	0	✓
漏报率	0	0	✓
审计链完整性	校验通过	通过	✓

7.6.3 性能优化措施

系统采用四项优化措施控制检测开销，见 表 7.9。其中指纹缓存的效果最为显著：重复语句的检测时延降至 0.08 ms 量级。

表 7.9 性能优化措施及其效果

优化点	措施	效果
语法分析开销	语句到指纹的最近最少使用缓存	重复语句降至 0.08 ms
基线查询	内存哈希表，启动时全量加载	常数时间查找
事件上报	异步队列与批量提交	不阻塞业务线程
差分计算	仅在指纹失配时触发	正常路径零额外开销

7.7 结果分析与讨论

7.7.1 关于第一轮中的五项未复现用例

表 7.5 中，TC-04、TC-05、TC-07、TC-10 与 TC-11 在第一轮未标记为复现。需要说明的是，这并不意味着相应漏洞不存在，而是靶场对该类载荷返回了错误响应而非成功执行的结果。

以 TC-05（时间盲注）为例：载荷 1' AND SLEEP(2) -- 在拼接后构成语法有效的语句并被数据库接受，注入客观成立；但由于测试所用的嵌入式数据库对延时函数的支持与生产数据

库存在差异，响应未呈现预期的延迟特征，判定脚本据此未标记为复现。类似情形亦见于其余四项。

这一现象揭示了漏洞验证中的一个方法论要点：判定漏洞是否存在的依据应是攻击载荷能否改变程序的执行语义，而非其是否产生了特定的可观测效果。后者受运行环境影响，可能造成对漏洞存在性的低估。第二轮中这五项用例均被成功拦截，恰说明检测引擎识别出了载荷的注入语义。

7.7.2 关于业务逻辑类漏洞的防线归属

七项业务逻辑类用例在第二轮标记为不适用，这是架构设计的必然结果而非缺陷。以水平越权为例：网关观测到请求 `GET /api/notes/detail?id=4`，该请求是否越权取决于“当前会话主体是否为该资源的属主”——这一信息存在于应用的会话状态中，网关不可见。

同理，服务端请求伪造的判定需要域名解析结果，令牌算法混淆的判定需要服务端密钥配置。此类漏洞的正确防线是应用代码层，本文的靶场安全实现已提供相应修复并在第三轮验证有效。

明确各类漏洞的适当防线，避免将所有防护责任压给单一组件，是安全架构设计的基本要求。将业务逻辑漏洞的防护强行下沉至网关，在技术上要求网关理解应用的全部业务语义，既不现实也不可维护。

8 项目管理

8.1 开发流程与进度安排

项目按课程实践指导书规定的四个阶段推进，实际执行的日程安排见图 8.1。图中标注了两个关键节点：技术风险验证前置于第三日完成，核心算法在第六日形成可验证的实现。

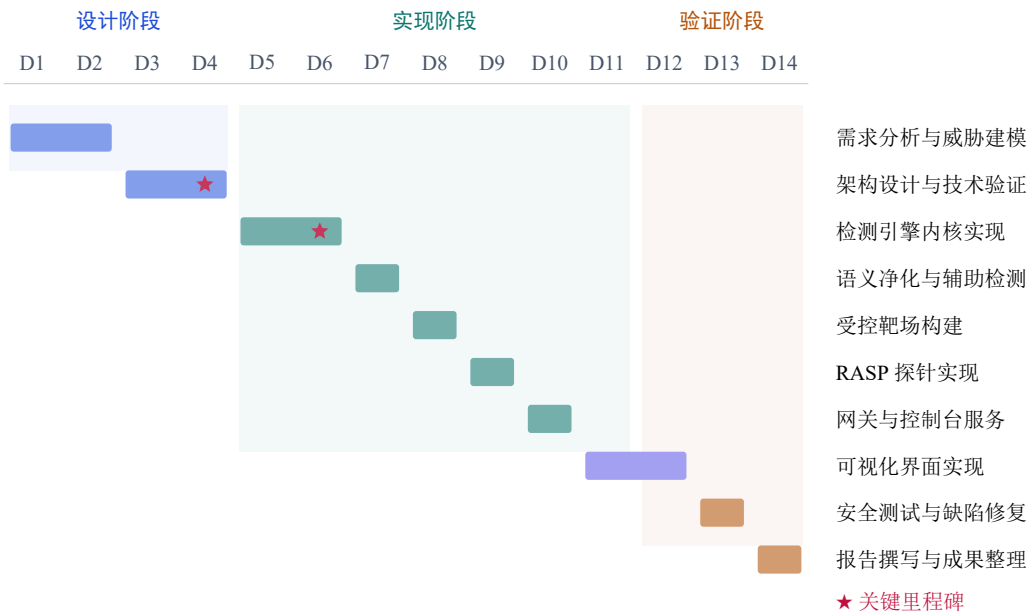


图 8.1 项目开发进度甘特图

8.2 风险管理

项目启动时识别了六项风险并预设应对措施，实际发生情况见 表 8.1。其中字节码注入的技术不确定性被判定为最高风险，故将其验证前置至第三日，并预设了退化方案。

表 8.1 风险登记表与实际发生情况

风险项	概率	影响	应对措施	实际情况
字节码注入受阻	中	高	退化为数据访问层显式埋点	未发生，一次验证通过
探针依赖冲突	中	高	打包时排除日志实现	发生并修复
语法解析方言差异	高	中	容错解析加词法层降级	发生，按预案处理
图渲染性能不足	低	中	限制节点数并支持折叠	未发生
进度延误	中	高	按优先级裁剪增强功能	未发生
误报率超标	中	中	提高相似度容差与接口豁免	发生并修复

8.3 工时统计

各阶段的工时投入见 图 8.2。实现阶段占比最高，其中检测引擎内核与可视化界面各占实现工时的三成左右。测试与缺陷修复的工时占比达 18%，这一比例反映了安全类项目对验证的高要求。

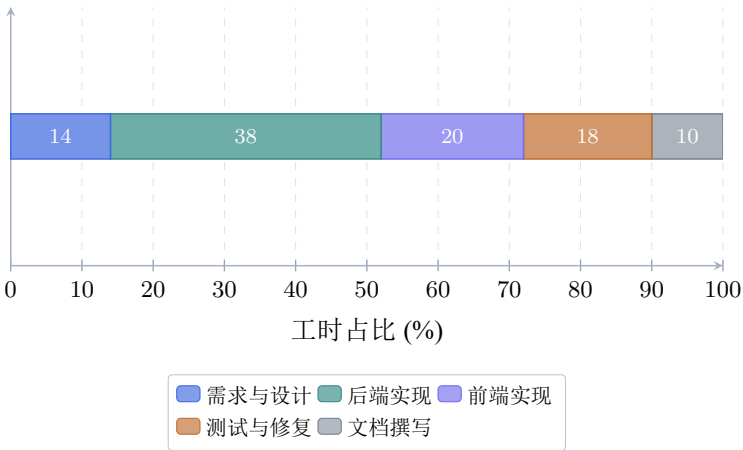


图 8.2 各阶段工时投入占比

8.4 小组成员与分工

项目由四名成员协作完成。分工遵循两条原则：其一，按子系统划分职责，使每位成员对所负责模块拥有完整的设计、实现与测试所有权，避免多人交叉修改同一模块引入的协调成本；其二，核心算法与总体架构由组长统一负责，以保证各模块在接口约定与判定逻辑上的一致性。

各成员的职责范围与主要产出见表 8.2。

表 8.2 小组成员分工与主要产出

成员	角色	职责范围	主要产出
黄思凯	组长	选题论证与总体方案设计；结构指纹与结构差分算法的设计与实现；双层架构设计与 RASP 探针实现；三轮测试方法设计；报告统稿	检测引擎内核、探针模块、算法形式化定义
罗锦皓	组员	安全网关的设计与实现：反向代理、检测过滤器链、参数判定的工程化；令牌桶与滑动窗口限流、来源信誉评估；性能测试	网关模块、限流与信誉组件、性能测试报告
曾铭	组员	受控靶场的构建：业务功能实现、漏洞与安全双实现对照；可视化前端：态势总览、语法树差分渲染、攻防演练台	靶场模块、前端五个功能页面
吴靖翔	组员	控制台服务：事件汇聚、哈希链审计、统计聚合与实时推送；OWASP 测试用例的执行、结果记录与回归测试补充	控制台模块、测试脚本与测试记录

各成员共同参与需求评审、代码互审与联调测试；报告由组长统稿，各成员分别撰写本人所负责模块的初稿。

协作流程上，项目采用“每日对齐、集中联调”的工作方式：实现阶段每日简短对齐各自进度与阻塞点；模块间接口（事件上报格式、追踪标识约定、策略同步协议）在编码前以文档形式冻结，之后各模块并行开发；集成阶段集中联调，期间发现并修复的跨模块缺陷均记录于开发日志。

模块负责人对其产出的正确性负第一责任，关键设计决策（如探针钩取点的选择、哈希链字段的构成）经组内讨论后由组长裁定，保证了决策效率与方案一致性的平衡。

8.5 版本控制与过程留痕

项目全程使用 Git 进行版本管理，遵循约定式提交规范，提交信息按 feat、fix、docs、test、chore 分类。代码托管于公开仓库，包含完整的提交历史、设计文档与测试脚本，构成可追溯的开发过程记录。

8.6 质量保障措施

项目采取三项质量保障措施：其一，检测引擎的每项算法均配套单元测试，且测试用例直接对应算法的形式化性质（如指纹的三条性质各有对应测试）；其二，每次发现缺陷后补充相应的回归测试，防止问题重现——本文的命令注入误报修复即配套新增了 12 项测试；其三，以三轮测试法进行端到端验证，避免单点验证的片面性。

9 总结与展望

9.1 工作总结

本文针对当前 Web 应用防火墙普遍采用的正则匹配范式“以文本形态为判定依据”这一方法论缺陷，提出并实现了基于抽象语法树结构指纹的语义检测方法，完成了从理论构造到工程系统的完整工作。主要工作归纳如下。

在方法层面，本文形式化定义了 SQL 结构指纹，并证明其同时满足参数无关性、形态无关性与结构敏感性三条性质。这三条性质的正交组合构成了本方法的理论基础：攻击者可自由操纵的文本形态不影响指纹，而攻击者必须改变的语义结构必然导致指纹失配。据此，编码混淆类绕过在原理上失效，而非依赖对已知手法的枚举。针对现有语法树方案“只能判定是否变化、无法定位变化位置”的不足，本文提出路径签名多重集差分算法，以线性复杂度实现结构级定位，并将差分结果映射为具有安全语义的攻击分类。

在系统层面，本文设计了网关与 RASP 探针的双层联动架构。该设计的依据是两层在可见性上的本质差异：网关仅能观测参数文本，判定必然是启发式的；探针可观测已完成拼接的真实语句，判定是确定性的。两层通过统一追踪标识关联，完整还原攻击证据链。系统另包含基于文档对象模型的跨站脚本语义净化、双重速率限制、带时间衰减的来源信誉评估与哈希链式防篡改审计日志。

在验证层面，本文采用三轮测试法分离各层防护的独立贡献，设计 19 项测试用例覆盖 OWASP Top 10 的七个类别。结果表明：网关可判定类攻击的实时拦截率为 12/12；全部 19 项漏洞的代码级修复验证有效；18 项高危漏洞全部获得防护；检测引擎单元测试通过率 63/63；平均检测时延 0.93 ms，优于设计指标。绕过对抗实验显示，七种文本变形的结构指纹完全一致，均被稳定检出。

9.2 创新点总结

1. **结构指纹方法的形式化构造与性质证明。**将“语义结构与文本形态正交”这一直觉转化为可证明的数学性质，使方法的有效性建立在原理之上而非经验之上。
2. **线性复杂度的结构差分定位算法。**以路径签名多重集差分替代复杂度过高的树编辑距离，在满足安全检测实际需求的前提下将复杂度降至 $O(n)$ ，并首次为语法树类检测方案提供了结构级的定位能力。
3. **基于可见性差异的双层架构论证。**明确区分两层的判定性质，避免了将确定性要求强加于信息不足的网关层，也避免了将拦截时效性要求强加于位置靠后的探针层。
4. **检测过程的完整可解释输出。**每次拦截均给出结构证据：变异节点的语法路径、语义分类、指纹对照与相似度量，使安全运营人员能够理解判定依据而非仅接受一个分值。

9.3 局限性分析

诚实记 □□ 方法的边界，是评估其适用范围的前提。本文识别出以下五项局限。

对语句可解析性的依赖。 结构指纹的构造以成功解析为前提。当遇到本文解析器未覆盖的数据库方言时，系统降级为词法层特征检测，此时结构级的定位能力丧失。改进方向是引入多解析器投票机制，或基于形式语法自动生成方言解析器。

基线质量对检测精度的制约。 若学习期内混入攻击流量，其结构将被误记为合法基线，导致同类攻击被放行。本文以人工审核环节缓解该风险，但审核本身依赖运营人员的判断力。改进方向是引入离群点检测，对显著偏离多数样本的候选基线自动标记。

二阶注入的溯源缺失。 攻击载荷先写入存储、在后续查询时才触发的二阶注入，本文可在触发时刻检出结构异常，但无法溯源至最初的写入请求。完整解决需要跨请求的数据流污点标记，这将显著增加系统复杂度。

业务逻辑漏洞的防护边界。 越权、服务端请求伪造等漏洞的判定需要会话状态或后端上下文，网关层不可见。本文明确将其归属于应用代码层防线，这一划分是合理的架构决策，但也意味着系统的防护范围存在明确边界。

加密流量的可见性要求。 网关须作为传输层安全的终止点方能检测流量内容。在端到端加密的部署形态下，网关层防护失效，此时仅探针层可用。

9.4 未来改进方向

基于上述局限，本文提出四个方向的改进设想。

其一，**跨请求的污点追踪。** 将数据的来源标记随其在系统中的流转持续传播，使二阶注入可被溯源至最初的污染点。技术路径是扩展 RASP 探针的钩取范围，在数据写入与读取时分别施加标记与检查。

其二，**基线的结构泛化。** 当前基线以精确的结构指纹匹配，对高度动态的查询（如可变列的报表查询）会产生大量合法指纹。改进方向是引入结构模板，将可变部分抽象为通配子树，提升基线的表达能力。

其三，**检测能力的横向扩展。** 本文的方法论——以语义结构而非文本形态作为判定依据——不限于 SQL。同样的思路可应用于表达式语言注入、模板注入等其他以“改变解释器语义”为目标的攻击类型。

其四，**与静态分析的结合。** 通过静态分析预先推导各查询点可能产生的语句结构集合，可减少运行时学习的样本需求，并为基线提供独立的交叉验证来源。

9.5 成员工作总结与心得

9.5.1 组长 黄思凯

两周的开发过程中，有三点体会值得记录。

方法论的选择先于技术的实现。 项目初期最重要的决策不是选用何种框架，而是确定“以什么作为检测依据”。一旦确立了“语义结构正交于文本形态”这一判据，后续的设计、架构划分乃至误报控制策略都获得了统一的评判标准。反之，若从“如何匹配更多攻击特征”出发，则会陷入规则堆砌的老路。这让我理解到，工程中的“难”往往不在于实现的复杂度，而在于是否找到了正确的问题表述。

自测是发现设计缺陷的唯一途径。 项目在设计文档阶段自认为方案已足够严密，但实际运行后仍暴露出多处问题：探针的异常抑制配置吞掉了拦截决策，导致“能检出却无法阻断”；参数探测法因构造脱离真实场景而将普通文本误判为攻击；哈希链因清空操作未同步重置内存状态而永久断裂。这些缺陷无一能通过纸面推演发现。这一经历让我对“测试驱动”有了具体而非口号式的理解——测试不是开发完成后的验收环节，而是发现设计缺陷的主要手段。

安全设计需要诚实面对边界。 在整理测试结果时，我们一度考虑将业务逻辑类漏洞的“网关未拦截”表述得更为含蓄，以使数据更好看。但最终选择了明确标注防线归属，并在报告中专门讨论这一架构决策的合理性。安全工程中，清楚地知道系统防不住什么，与知道它能防住什么同等重要。一份掩盖边界的防护方案，其危害在于会诱导使用者产生不实的安全感，而这恰是安全事故的常见诱因。

作为组长，我还体会到分工与一致性的平衡之道：把子系统完整地交给每位成员独立负责，能激发各自的理解与责任心；但判定逻辑这类牵一发而动全身的核心约定，必须有人端到端地把关。本次项目的顺利推进得益于这种“模块自治、核心集中”的组织方式。

9.5.2 组员 罗锦皓

我负责安全网关与限流模块的实现。最大的收获是对“启发式判定边界”的理解：网关只能看到参数文本，无法确知它会被拼接到何处，这决定了网关层的判定在原理上就是概率性的。实现参数检测时，我最初将参数裸拼进模板语句解析，导致普通文本被误判为攻击；在与组长讨论后改为将参数作为字符串字面量注入探针，模拟真实的拼接场景，误报随之消除。这让我认识到检测方法的构造必须与被检测场景的真实语义保持一致，而不能想当然。

9.5.3 组员 曾铭

我负责受控靶场与可视化前端。构建漏洞与安全双实现的过程，迫使我从攻击者的视角逐个理解每处漏洞的成因——为什么闭合一个引号就能绕过认证，为什么一个 `onerror` 属性就能窃取会话。这种理解深度是单纯阅读教材无法获得的。前端部分，语法树差分渲染的调试让我体会到“动效服务于信息”这一原则的意义：注入节点的脉冲提示之所以有冲击力，正因为它指向了判定结论本身，而非单纯的视觉装饰。

9.5.4 组员 吴靖翔

我负责控制台服务与安全测试的执行。印象最深的是三轮测试中发现的哈希链断裂问题：清空事件后新记录的前置哈希仍指向已删除的记录，完整性校验从此永久失败。定位并修复这个缺陷让我具体理解了“持久化状态与内存状态必须同步”这类工程细节的重要性。此外，执

行 19 项用例的三轮验证也让我体会到规范化测试记录的价值——正是完整的每轮结果对照，使各类漏洞的防线归属得以清晰呈现。

通过本次实践，全体成员共同建立了从威胁建模、方案论证到实验验证的完整方法认知，并对“安全第一、预防为主”这一原则形成了基于实践的理解。

参考文献

- [1] OWASP Foundation. *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*. Tech. rep. Open Worldwide Application Security Project, 2021.
- [2] OWASP Foundation. *OWASP ModSecurity Core Rule Set: Documentation and Rule Reference*. Tech. rep. Open Worldwide Application Security Project, 2023.
- [3] B. Cui, J. Zhang, and Z. Cui. “A Survey on SQL Injection Attack Detection and Prevention Techniques”. In: *Journal of Network and Computer Applications* 121 (2018), pp. 1–18.
- [4] N. Galbreath. *libinjection: A C Library for SQLi and XSS Detection Using Lexical Analysis*. Black Hat USA Technical Briefing, 2012.
- [5] G. Buehrer, B. W. Weide, and P. A. G. Sivilotti. “Using Parse Tree Validation to Prevent SQL Injection Attacks”. In: *Proceedings of the 5th International Workshop on Software Engineering and Middleware (SEM)*. Lisbon, Portugal: ACM, 2005, pp. 106–113.
- [6] S. W. Boyd and A. D. Keromytis. “SQLrand: Preventing SQL Injection Attacks”. In: *Proceedings of the 2nd International Conference on Applied Cryptography and Network Security (ACNS)*. Vol. 3089. Lecture Notes in Computer Science. Springer, 2004, pp. 292–302.
- [7] W. G. J. Halfond and A. Orso. “AMNESIA: Analysis and Monitoring for Neutralizing SQL-Injection Attacks”. In: *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2005, pp. 174–183.
- [8] Z. Su and G. Wassermann. “The Essence of Command Injection Attacks in Web Applications”. In: *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2006, pp. 372–382.
- [9] A. Luo, W. Huang, and W. Fan. “A CNN-Based Approach to the Detection of SQL Injection Attacks”. In: *Proceedings of the IEEE/ACIS International Conference on Computer and Information Science (ICIS)* (2019), pp. 320–324.
- [10] P. Tang, W. Qiu, Z. Huang, H. Lian, and G. Liu. “Detection of SQL Injection Based on Artificial Neural Network”. In: *Knowledge-Based Systems* 190 (2020), p. 105528.
- [11] 李梦, 孙彦, and 周航. “SQL 注入攻击检测技术研究进展”. In: *通信学报* 41.11 (2020), pp. 189–203.
- [12] 王晓东, 刘建伟, and 赵鹏. “基于运行时应用自我保护的 Web 攻击检测技术综述”. In: *软件学报* 33.5 (2022), pp. 1682–1705.
- [13] K. Zhang and D. Shasha. “Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems”. In: *SIAM Journal on Computing* 18.6 (1989), pp. 1245–1262.
- [14] A. Shostack. *Threat Modeling: Designing for Security*. Book. 2014.
- [15] FIRST.Org. *Common Vulnerability Scoring System Version 3.1: Specification Document*. Tech. rep. Forum of Incident Response and Security Teams, 2019.
- [16] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Addison-Wesley, 2006.
- [17] R. Winterhalter. *Byte Buddy: Runtime Code Generation for the Java Virtual Machine*. <https://bytebuddy.net>. 2024.
- [18] P. Bille. “A Survey on Tree Edit Distance and Related Problems”. In: *Theoretical Computer Science* 337.1–3 (2005), pp. 217–239.

-
- [19] 全国信息安全标准化技术委员会. 信息安全技术 信息安全风险评估方法: *GB/T 20984-2022*. Tech. rep. 中国标准出版社, 2022.
 - [20] 全国信息安全标准化技术委员会. 信息安全技术 网络安全等级保护安全设计技术要求: *GB/T 25070-2019*. Tech. rep. 中国标准出版社, 2019.
 - [21] N. Provos and D. Mazières. *A Future-Adaptable Password Scheme*. Proceedings of the USENIX Annual Technical Conference. 1999.

附录

A 核心算法实现

本附录摘录检测引擎的两处核心实现。完整代码见项目仓库。

A.1 字面量归一 □□

该方法是结构指纹获得参数无关性的实现基础。其中两处判断值得注意：**IN** 列表的长度无关化避免了基线爆炸，函数名的保留避免了危险函数漏报。

```
1 private static void normalizeInPlace(AstNode node) {
2     switch (node.getType()) {
3         case LITERAL -> normalizeLiteral(node);
4
5         case IN_EXPRESSION -> {
6             // IN 列表长度无关化：先归一化子节点，再折叠重复占位符，
7             // 避免 IN (1,2) 与 IN (1,2,3) 产生不同指纹导致基线爆炸
8             for (AstNode child : node.mutableChildren()) {
9                 normalizeInPlace(child);
10            }
11            collapseInList(node);
12            return;
13        }
14
15        case FUNCTION_CALL -> {
16            // 函数名保留不变。SLEEP、LOAD_FILE 等是时间盲注与带外
17            // 注入的核心标志，若一并归一化将造成最危险一类攻击的漏报
18            for (AstNode child : node.mutableChildren()) {
19                normalizeInPlace(child);
20            }
21            return;
22        }
23
24        default -> { /* 表名、列名、运算符等结构性节点保持原样 */ }
25    }
26    for (AstNode child : node.mutableChildren()) {
27        normalizeInPlace(child);
28    }
29 }
```

代码附录 A-1 AST 字面量归一化的核心逻辑

A.2 结构指纹计算

```
1 public ParseResult analyze(String sql, String endpoint) {
2     // 缓存命中则直接返回，使重复语句的检测开销降至可忽略水平
3     ParseResult cached = cache.getIfPresent(endpoint + "\0" + sql);
4     if (cached != null) return cached;
5
6     ParseResult result;
7     try {
```

```

8      AstNode ast      = new SqlParser(sql).parse();
9      AstNode normalized = AstNormalizer.normalize(ast);
10     String canonical = AstSerializer.serialize(normalized);
11     // 指纹绑定接口标识，防止跨接口复用合法结构以绕过检测
12     String fingerprint = hash(endpoint + "|" + canonical);
13     result = new ParseResult(sql, endpoint, ast, normalized,
14                             canonical, fingerprint, true, null);
15 } catch (RuntimeException e) {
16     // 解析异常降级：仍产出可用结果交由风险特征层判定。
17     // 解析失败绝不能成为绕过手段，故此处不放行
18     result = new ParseResult(sql, endpoint, null, null, sql,
19                             hash(endpoint + "|" + sql), false,
20                             e.getMessage());
21 }
22 cache.put(endpoint + "\0" + sql, result);
23 return result;
24 }
```

代码 附录 A-2 结构指纹的计算流程

B 测试用例载荷

表 B-1 列出主要测试用例的攻击载荷。

使用限制 以下载荷仅用于本地隔离环境中针对自建靶场的安全测试。任何针对未授权系统的使用均属违法行为。

表 B-1 主要测试用例的攻击载荷

用例	载荷
TC-01	username = admin' OR '1'='1' --
TC-02	keyword = x%' UNION SELECT id, id, username, CAST(password AS VARCHAR), role, created_at FROM users --
TC-03	id = 1' OR '1'='1
TC-04	keyword = x%'/*!50000UNION*/SELECT ... FROM users --
TC-05	id = 1' AND SLEEP(2) --
TC-06	id = 1'; DROP TABLE notes_backup; --
TC-07	id = 1%2527%2520OR%2520%25271%2527%253D%25271
TC-08	content =
TC-09	content = <svg/onload=alert(1)>
TC-10	content = x
TC-11	host = 127.0.0.1; cat /etc/passwd
TC-14	name = ../../../../etc/passwd
TC-19	头部改为 {"alg": "none"}, 删除签名段并篡改角色声明
TC-22	url = http://127.0.0.1:8080/api/health

C 项目成果清单

表 C-1 项目交付成果

类别	成果	说明
源代码	后端六模块	约 1.09 万行 Java 代码
	前端应用	约 0.61 万行 Vue 代码
	测试脚本	单元测试与安全测试脚本
文档	设计文档	方案总览、架构设计、算法设计等六份
	本报告	综合设计报告
其他	代码仓库	含完整提交历史的公开仓库
	测试记录	三轮测试的完整执行结果

D 合规声明

本项目所有攻击测试均在本地隔离环境中针对自建靶场系统执行，未对任何第三方系统实施未授权测试。

靶场中的漏洞代码为教学目的主动构造，集中隔离于独立代码包并逐一加注警示注释，不随生产构建发布。反序列化类测试用例采用无害化载荷，不包含任何具备实际破坏能力的利用链。

本项目严格遵守《中华人民共和国网络安全法》《中华人民共和国数据安全法》及相关法律法规，并参照国家标准 GB/T 20984 关于信息安全风险评估活动的规范要求执行。所有技术成果仅用于安全防护研究与教学实践。